

User Manual

for S32K1_S32M24X I2S Driver

Document Number: UM2I2SASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	6
2.5 Reference List	6
3 Driver	7
3.1 Requirements	7
3.2 Driver Design Summary	7
3.3 Hardware Resources	7
3.4 Deviations from Requirements	8
3.5 Driver Limitations	8
3.6 Driver usage and configuration tips	8
3.6.1 SAI and Flexio I2S initializations	8
3.6.2 SAI and Flexio I2S data transmission	8
3.6.3 I2s over SAI DMA transfer	9
3.6.4 I2s over SAI full duplex mode	9
3.6.5 Flexio_I2s communicates with Sai	9
3.6.6 SAI auto disable clock	10
3.6.7 I2s Continuous transmission	10
3.7 Runtime errors	10
3.8 Symbolic Names Disclaimer	11
3.9 Usage in non AUTOSAR context	11
4 Configuration Parameters	13
4.1 I2s	14
4.1.1 Module I2s	16
4.1.2 Container GeneralConfiguration	17
4.1.3 Parameter I2sDevErrorDetect	17
4.1.4 Parameter I2sMultipartitionSupport	17
4.1.5 Parameter I2sEnableUserModeSupport	18
4.1.6 Parameter I2sVersionInfoApi	18
4.1.7 Parameter I2sTimeoutMethod	19
4.1.8 Parameter I2sTimeoutVal	19
4.1.9 Container I2sSaiSupport	20
4.1.10 Parameter I2sSaiEnable	20
4.1.11 Parameter SaiIpDevErrorDetect	20
4.1.12 Parameter SaiDmaFeature	21
4.1.13 Parameter SaiAutoDisableClockFeature	21
4.1.14 Parameter SaiFifoPackFeature	22
4.1.15 Parameter SaiBitClkSwapSupport	22
4.1.16 Parameter SaiEnableDataMasking	22
4.1.17 Container I2sFlexioSupport	23
4.1.18 Parameter I2sFlexioI2sEnable	23
4.1.19 Parameter FlexioI2sIpDevErrorDetect	24
4.1.20 Parameter FlexioI2sDmaFeature	24
4.1.21 Container I2sConfigSet	24
4.1.22 Container I2sConfiguration	25
4.1.23 Parameter I2sIpType	25
4.1.24 Parameter I2sLogicalChannelId	25
4.1.25 Parameter I2sTxCallback	26

4.1.26	Parameter	I2sRxCallback	26
4.1.27	Reference	I2sModuleRef	27
4.1.28	Reference	I2sChannelEcucPartitionRef	27
4.1.29	Container	SaiHwConfiguration	27
4.1.30	Parameter	SaiInstanceId	28
4.1.31	Parameter	SaiTxRxConfiguration	28
4.1.32	Parameter	SaiMode	29
4.1.33	Parameter	SaiMasterClkSrc	29
4.1.34	Parameter	SaiSyncType	29
4.1.35	Parameter	SaiBitClkSwap	30
4.1.36	Parameter	SaiBitClkNegPolar	30
4.1.37	Parameter	SaiSampleRate	31
4.1.38	Parameter	SaiDataLine0	31
4.1.39	Parameter	SaiDataLine1	32
4.1.40	Parameter	SaiDataLine2	32
4.1.41	Parameter	SaiDataLine3	32
4.1.42	Parameter	SaiFrameSize	33
4.1.43	Parameter	SaiSyncWidth	33
4.1.44	Parameter	SaiChannelMode	34
4.1.45	Parameter	SaiMsbFirst	34
4.1.46	Parameter	SaiEnableDebug	34
4.1.47	Parameter	SaiSyncEarly	35
4.1.48	Parameter	SaiSyncNegPolar	35
4.1.49	Parameter	SaiWord0Width	36
4.1.50	Parameter	SaiWordNWidth	36
4.1.51	Parameter	SaiFirstBitIndex	36
4.1.52	Parameter	SaiRunErrorReport	37
4.1.53	Parameter	SaiSyncErrorReport	37
4.1.54	Parameter	SaiFrameStartReport	38
4.1.55	Parameter	SaiFifoPackEnable	38
4.1.56	Parameter	SaiMuxMode	38
4.1.57	Parameter	SaiTransferType	39
4.1.58	Parameter	SaiElementSize	39
4.1.59	Parameter	SaiBufferAccessSize	40
4.1.60	Parameter	SaiDataLineCount	40
4.1.61	Parameter	SaiWordFlagIndex	41
4.1.62	Parameter	SaiBitClkInput	41
4.1.63	Parameter	SaiContOnErr	41
4.1.64	Parameter	SaiSyncOnDemand	42
4.1.65	Reference	SaiClkSrcRef	42
4.1.66	Reference	SaiDmaChannel0	43
4.1.67	Reference	SaiDmaChannel1	43
4.1.68	Reference	SaiDmaChannel2	43
4.1.69	Reference	SaiDmaChannel3	44
4.1.70	Container	FlexioHwConfiguration	44
4.1.71	Parameter	FlexioI2sInstanceId	45
4.1.72	Parameter	FlexioI2sBitsWidth	45
4.1.73	Parameter	FlexioI2sMode	45
4.1.74	Parameter	FlexioI2sBaudRate	46
4.1.75	Parameter	FlexioI2sTransferType	46
4.1.76	Parameter	FlexioI2sTxLine	47
4.1.77	Parameter	FlexioI2sRxLine	47
4.1.78	Parameter	FlexioI2sWsLine	47
4.1.79	Parameter	FlexioI2sClkSamplingEdge	48

4.1.80	Parameter FlexioI2sTxCallbackParam	48
4.1.81	Parameter FlexioI2sRxCallbackParam	49
4.1.82	Reference FlexioI2sClkSrcRef	49
4.1.83	Reference FlexioI2sDataLine	49
4.1.84	Reference FlexioI2sSckLine	50
4.1.85	Reference FlexioI2sDmaTxChannel	50
4.1.86	Reference FlexioI2sDmaRxChannel	51
4.1.87	Container CommonPublishedInformation	51
4.1.88	Parameter ArReleaseMajorVersion	51
4.1.89	Parameter ArReleaseMinorVersion	52
4.1.90	Parameter ArReleaseRevisionVersion	52
4.1.91	Parameter ModuleId	53
4.1.92	Parameter SwMajorVersion	53
4.1.93	Parameter SwMinorVersion	54
4.1.94	Parameter SwPatchVersion	54
4.1.95	Parameter VendorApiInfix	54
4.1.96	Parameter VendorId	55
4.1.97	Container I2sClockReferencePoint	55
4.1.98	Reference I2sClockReference	56
4.2	SAI_IP	56
4.2.1	Module Sai_Ip	56
4.2.2	Parameter SaiEnableUserModeSupport	57
4.2.3	Parameter SaiTimeoutMethod	57
4.2.4	Parameter SaiTimeoutVal	58
4.3	FLEXIO_I2S_IP	58
4.3.1	Module Flexio_I2s_Ip	59
4.3.2	Parameter FlexioI2sEnableUserModeSupport	59
4.3.3	Parameter FlexioI2sTimeoutMethod	60
4.3.4	Parameter FlexioI2sTimeoutVal	60
5	Module Index	62
5.1	Software Specification	62
6	Module Documentation	63
6.1	I2s Driver	63
6.1.1	Detailed Description	63
6.1.2	Data Structure Documentation	65
6.1.2.1	struct I2s_StatusStructType	65
6.1.2.2	struct I2s_ConfigType	65
6.1.2.3	struct I2s_RequestType	65
6.1.2.4	struct I2s_Ipw_ConfigType	66
6.1.2.5	struct I2s_HwConfigType	66
6.1.2.6	struct I2s_LogicalConfigState	66
6.1.3	Macro Definition Documentation	66
6.1.3.1	I2S_E_UNINIT	66
6.1.3.2	I2S_E_BUSY	66
6.1.3.3	I2S_E_TIMEOUT	67
6.1.3.4	I2S_E_INIT_FAILED	67
6.1.3.5	I2S_E_PARAM_POINTER	67
6.1.3.6	I2S_E_PARAM_CHANNEL	67
6.1.3.7	I2S_E_PARAM_CONFIG	67
6.1.3.8	I2S_E_INCORRECT_DIRECTION	67
6.1.3.9	I2S_INIT_ID	68
6.1.3.10	I2S_DEINIT_ID	68

6.1.3.11 I2S_SYNCTRANSMIT_ID	68
6.1.3.12 I2S_ASYNCTRANSMIT_ID	68
6.1.3.13 I2S_GETSTATUS_ID	68
6.1.3.14 I2S_ABORTTRANSMIT_ID	68
6.1.3.15 I2S_GETVERSIONINFO_ID	69
6.1.4 Types Reference	69
6.1.4.1 I2s_HwUnitCount	69
6.1.4.2 I2s_DataType	69
6.1.4.3 I2s_WordMaskType	69
6.1.4.4 I2s_LogicalChannel	69
6.1.5 Enum Reference	69
6.1.5.1 I2s_DataDirectionType	69
6.1.5.2 I2s_StatusType	70
6.1.5.3 I2s_TransmitReceiveType	70
6.1.5.4 I2s_ModeType	70
6.1.5.5 I2s_SupportedIPsType	71
6.1.6 Function Reference	71
6.1.6.1 I2s_Init()	71
6.1.6.2 I2s_DeInit()	71
6.1.6.3 I2s_SyncTransmit()	72
6.1.6.4 I2s_AsyncTransmit()	72
6.1.6.5 I2s_GetStatus()	73
6.1.6.6 I2s_AbortTransmit()	73
6.1.6.7 I2s_GetVersionInfo()	74
6.2 FLEXIO_I2S IPL	75
6.2.1 Detailed Description	75
6.2.2 Data Structure Documentation	76
6.2.2.1 struct Flexio_I2s_Ip_CommonStateType	76
6.2.2.2 struct Flexio_I2s_Ip_StateType	76
6.2.2.3 struct Flexio_I2s_Ip_MasterConfigType	76
6.2.2.4 struct Flexio_I2s_Ip_SlaveConfigType	77
6.2.3 Enum Reference	78
6.2.3.1 Flexio_I2s_Ip_StatusType	78
6.2.3.2 Flexio_I2s_Ip_DriverType	78
6.2.3.3 Flexio_I2s_Ip_EventType	78
6.2.4 Function Reference	79
6.2.4.1 Flexio_I2s_Ip_MasterInit()	79
6.2.4.2 Flexio_I2s_Ip_Deinit()	79
6.2.4.3 Flexio_I2s_Ip_SendData()	80
6.2.4.4 Flexio_I2s_Ip_SendDataBlocking()	80
6.2.4.5 Flexio_I2s_Ip_ReceiveData()	81
6.2.4.6 Flexio_I2s_Ip_ReceiveDataBlocking()	81
6.2.4.7 Flexio_I2s_Ip_TransferAbort()	82
6.2.4.8 Flexio_I2s_Ip_GetStatus()	82
6.2.4.9 Flexio_I2s_Ip_SlaveInit()	83
6.2.4.10 Flexio_I2s_Ip_IrqHandler()	83
6.3 SAI IPL	84
6.3.1 Detailed Description	84
6.3.2 Data Structure Documentation	86
6.3.2.1 struct Sai_Ip_XferState	86
6.3.2.2 struct Sai_Ip_StateStructType	86
6.3.2.3 struct Sai_Ip_ConfigType	86
6.3.3 Macro Definition Documentation	88
6.3.3.1 SAI_IP_FPACK_8_BIT	88

6.3.3.2	SAI_IP_FPACK_16_BIT	88
6.3.3.3	FEATURE_SAI_IP_HAS_CHMOD	88
6.3.3.4	FEATURE_SAI_IP_MSEL_BUS_CLK	88
6.3.3.5	FEATURE_SAI_IP_MSEL_SOSC_DIV1_CLK	88
6.3.3.6	FEATURE_SAI_IP_MSEL_MCLK_PIN	89
6.3.3.7	FEATURE_SAI_IP_MSEL_OTHER_MCLK_PIN	89
6.3.3.8	FEATURE_SAI_IP_HAS_PARAM	89
6.3.3.9	SAI0_IP_CHANNEL_COUNT	89
6.3.3.10	SAI_IP_NULL_PTRS	89
6.3.4	Types Reference	89
6.3.4.1	Sai_Ip_WriteHandlerType	89
6.3.4.2	Sai_Ip_ReadHandlerType	90
6.3.4.3	Sai_Ip_TransferCallbackType	90
6.3.5	Enum Reference	90
6.3.5.1	Sai_Ip_ReportType	90
6.3.5.2	Sai_Ip_StatusType	90
6.3.5.3	Sai_Ip_TransferType	91
6.3.5.4	Sai_Ip_MuxType	91
6.3.5.5	Sai_Ip_SyncType	91
6.3.5.6	Sai_Ip_MasterClkSelType	92
6.3.5.7	Sai_Ip_MaskType	92
6.3.6	Function Reference	92
6.3.6.1	Sai_Ip_SetMaster()	92
6.3.6.2	Sai_Ip_TxInit()	93
6.3.6.3	Sai_Ip_RxInit()	93
6.3.6.4	Sai_Ip_TxDeInit()	93
6.3.6.5	Sai_Ip_RxDeInit()	94
6.3.6.6	Sai_Ip_SendBlocking()	94
6.3.6.7	Sai_Ip_Send()	94
6.3.6.8	Sai_Ip_GetSendingStatus()	95
6.3.6.9	Sai_Ip_ReceiveBlocking()	95
6.3.6.10	Sai_Ip_Receive()	96
6.3.6.11	Sai_Ip_GetReceivingStatus()	96
6.3.6.12	Sai_Ip_AbortTransfer()	97
6.3.6.13	Sai_Ip_IRQHandler()	97
6.3.6.14	Sai_Ip_TxAutoDisableClock()	97
6.3.6.15	Sai_Ip_RxAutoDisableClock()	97

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes the NXP Semiconductor I2s driver for S32K1xx_S32M24x. I2s driver configuration parameters and deviations from the specification are described in Driver chapter of this document. I2s driver requirements and APIs are described in the I2s driver software specification document.

Note: The Sai driver has been renamed to I2s starting with release RTD S32K1 Version 1.0.1. The API of the driver maintains the same functionality as previous releases. However, API names have been changed, beginning with the prefix "I2s_" instead of "Sai_". Existing application code should be updated to be in accordance with the new API naming.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100

- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64
- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CDD	Complex Device Driver
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
I2S	Inter-IC Sound
SIU	Systems Integration Unit
SWS	Software Specification
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

I2s is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The Inter-IC Sound (I2S) driver is implemented as an Autosar complex device driver and provides an interface that supports full-duplex and half-duplex serial interfaces with frame synchronization such as I2S, AC97, TDM, and codec/DSP interfaces. The driver uses the SAI and Flexio I2S peripherals in S32K1XX microcontrollers. The driver can be used to configure the SAI and Flexio I2S peripherals and initiate synchronous and asynchronous data transfers. The peripherals can be configured to operate as a receiver or transmitter, operating in slave and master mode. Hardware and software settings can be configured using an Autosar standard configuration tool. The information required for a I2s data transfer will be configured in a data structure that will be sent as parameter to the API of the driver.

3.3 Hardware Resources

The I2s driver uses the SAI and Flexio I2S hardware IPs. For more details and availability please check the device reference manual.

Note

S32M244 device only has support for FlexIO I2S, so any information regarding SAI peripheral in this documentation should be disregarded.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR CDD_I2S Driver software specification in some places. The table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the I2S Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, not available, not testable or out of scope for the driver.

Requirement	Status	Description	Notes
Req_Id	N/S	These requirements are not applicable to this specification.	Not a requirement.

3.5 Driver Limitations

I2s driver limitations:

1. I2S over SAI FIFO packing with MUXLINE/MUXDMA feature is not supported for DMA transfer mode.
2. I2S with DMA transfer counts limited by 32767 without assert or warning by driver(Dma Hw limitation).
3. I2S over FlexIO DMA TX: When transmitting over DMA the driver stops transmission 2 frames early. Workaround: Setup buffer of <desired_buffer_length + 2> and setup the transmit request bufferSize with value <desired_buffer_size + 2> before calling [I2s_AsyncTransmit\(\)](#).
4. Only in EB configuration for each I2s logical channel both Rx and Tx paths are mandatory. In S32DS this is not valid, Rx or Tx can be configured independently as needed

3.6 Driver usage and configuration tips

3.6.1 SAI and Flexio I2S initializations

In order to use the I2S driver it must be first initialized in transmit/receive for SAI and master/slave for Flexio I2S for each I2S logical channel that mapping with the desired hardware unit using functions [I2s_Init\(\)](#). Once initialized, it cannot be initialized again until it is de-initialized using [I2s_DeInit\(\)](#).

3.6.2 SAI and Flexio I2S data transmission

In each mode (transmit/receive for SAI and master/slave for Flexio I2S) are available two types of transfers:

- Synchronous (blocking): the functions which initiate blocking transfers with configured timeout for transmission. If time expires [I2s_SyncTransmit\(\)](#) will return error and the transmission will be aborted.

- Asynchronous (non-blocking): `I2s_AsyncTransmit()` will return status immediately and complete after the last data is transferred.

Uses `I2s_AbortTransmit()` to abort or stop the ongoing transfer.

Uses `I2s_GetStatus()` to get the information of the current transfer.

Uses `I2s_GetVersionInfo()` to get version information of this module.

3.6.3 I2s over SAI DMA transfer

- DMA module has to be initialized prior to usage in DMA mode; also, DMA channels need to be allocated by the application (the driver only takes care of configuring the DMA channels received in the configuration structure)
- There is a difference in ChannelEnable field usage between interrupt and dma mode:
In interrupt mode, if mux line is enabled then user must turn on only one bit in ChannelEnable, which will be the data line to output data. Number of data buffers to be muxed is specified in ChannelCount field.
In DMA mode, if mux line is enabled then user must turn on number of bits equal to number of data buffers to be muxed. The data lines corresponding to these bits will output the same as each other.
Also in DMA mode, if a mux mode is selected, ChannelEnable must be turned on from bit 0, and immediately above (for example turning on bit 0 and bit 2 is not a correct configuration).
- When multiple SAI channel is enabled, number of DMA channels must be equal number of SAI channels. And in DMA component:
Channel arbitration must be set to "Fixed priority".
The first DMA channel must have the lowest DMA channel number and request source must be set properly (for example SAI0_TX). Other DMA channels must have request field set to "No request".

3.6.4 I2s over SAI full duplex mode

- Configuration:
 - Initialize both Transmitter and Receiver with one is Master-Asynchronous and another is Slave- $\leftrightarrow$ Synchronous.
 - Setup different data lines between Transmitter and Receiver (BCLK and FS will be shared).
 - The SAI protocol configuration must be the same between Transmitter and Receiver.
 - Transmit data pins are configured for TDM mode (MASK_TRISTATE - transmit data pins are tri-stated when slots are masked or channels are disabled).
- Transmission:
 - Slave request (non-blocking) should be called first (to prepare data transmission) then Master request shall start the transmission.
 - Master request can be used alone if do not need any action from Slave.

3.6.5 Flexio_I2s communicates with Sai

- Flexio_I2s is Master and Sai is Slave:
 - For Sai, A valid frame sync is also ignored (slave mode) or not generated (master mode) for the first 4 bit clocks but Flexio_I2s does not

- When Sai receives data from Flexio_I2s, it ignores the falling/rising edge on WS line during the 4 first bit clocks until next falling/rising edge, so depending on Frame Sync polarity, the actual received buffer (by Sai) is less than the transferred buffer size (from Flexio_I2s) 1 or 2 words
- When Sai transmits data, Sai also ignores the falling/rising edge on WS line and does not generate DATA during 4 first bit clocks until next falling/rising edge Flexio_I2s needs to generate 1 more frame clock, the actual received buffer (by Flexio_i2s) is greater than the transferred buffer (from Sai) 2 words
- Sai is Master and Flexio_I2s is Slave:
 - The above issue does not happen when Sai is master and Flexio_I2s is slave, because the first 4 bit clocks without WS edge do not affect Flexio_I2s, transmission start is just delayed with 4 bit clocks.

3.6.6 SAI auto disable clock

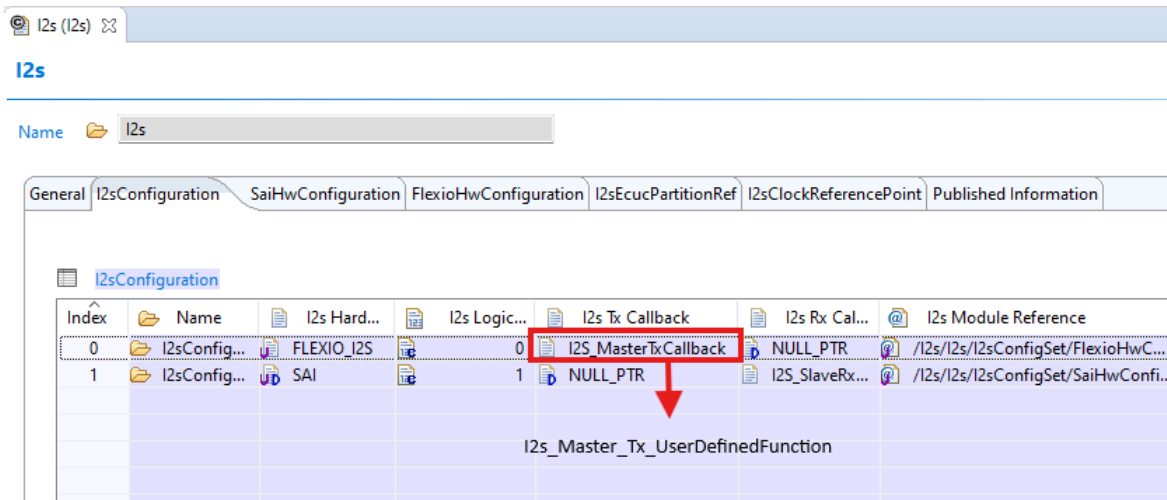
If there is no more request (transmit/receive) added in a long time after, "autoDisableClock" can be set in the last request to disable clock generation until the next request.

3.6.7 I2s Continuous transmission

For a continuous data transfer on I2S a new transfer must be initiated in the I2S callback function. The callback functions can be enabled on each I2S Logical channel for both Rx and Tx

If enabled, the UseDefinedFunction callback will be executed when the previous transfer is completed, and in this mechanism the user must start a new transmission, thus obtaining a continuous data transmission.

Ex: Master Tx Callback



Limitation: Restarting a transmission is possible, however, there is a time lag between the last frame of the previous transfer and the first frame of the new transfer. This is due to the time consumed by the Callback, needed to initialize a new transfer. This lag depends on the specific configuration of the project (DMA/Irq, clock) During this time frames will be lost (the last valid frame will be sent) The registers are not yet updated with the new values via Dma, so the old values will be sent until the transmission is restarted

Ex: For an I2S channel configuration with Dma, SystemClock 112 MHz reinitializing a transmission takes about 42 nanoseconds

3.7 Runtime errors

The driver generates the following DET runtime errors at runtime.

Function	Error Code	Condition triggering the error
I2s_SyncTransmit()	I2S_E_TIMEOUT	Timed out. Ongoing transmission timed out.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

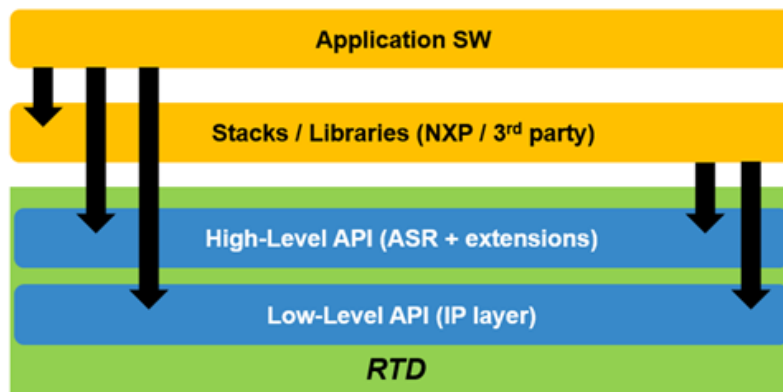


Figure 3.1 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers

5. Runtime functionality

6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

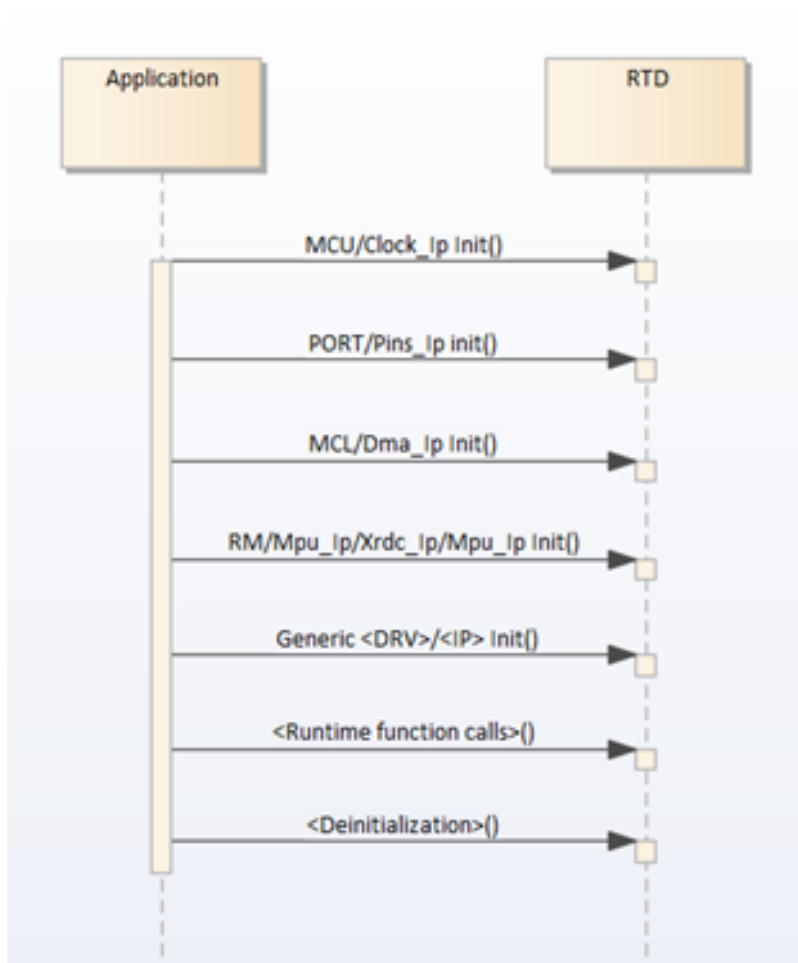


Figure 3.2 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

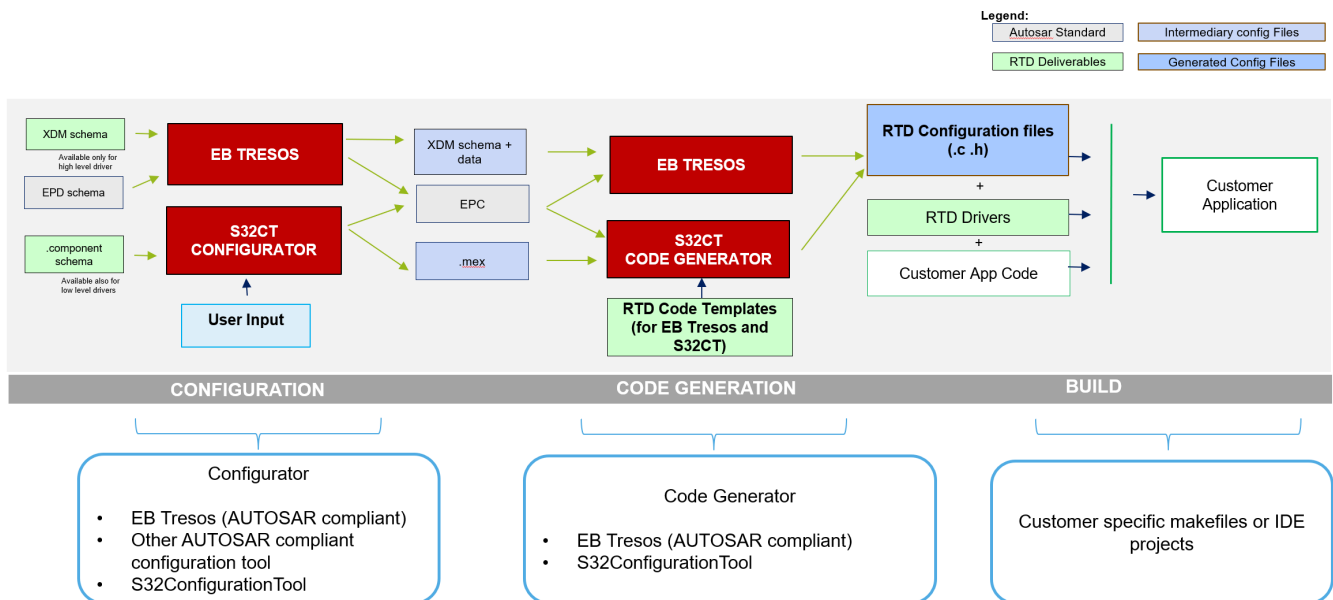


Figure 4.1 Configuration generation flow

- I2s
- SAI_IP
- FLEXIO_I2S_IP

4.1 I2s

- Module [I2s](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [I2sDevErrorDetect](#)
 - * Parameter [I2sMultipartitionSupport](#)
 - * Parameter [I2sEnableUserModeSupport](#)
 - * Parameter [I2sVersionInfoApi](#)
 - * Parameter [I2sTimeoutMethod](#)
 - * Parameter [I2sTimeoutVal](#)
 - * Container [I2sSaiSupport](#)
 - Parameter [I2sSaiEnable](#)
 - Parameter [SaiIpDevErrorDetect](#)
 - Parameter [SaiDmaFeature](#)
 - Parameter [SaiAutoDisableClockFeature](#)
 - Parameter [SaiFifoPackFeature](#)
 - Parameter [SaiBitClkSwapSupport](#)
 - Parameter [SaiEnableDataMasking](#)
 - * Container [I2sFlexioSupport](#)
 - Parameter [I2sFlexioI2sEnable](#)
 - Parameter [FlexioI2sIpDevErrorDetect](#)
 - Parameter [FlexioI2sDmaFeature](#)
 - Container [I2sConfigSet](#)
 - * Container [I2sConfiguration](#)
 - Parameter [I2sIpType](#)
 - Parameter [I2sLogicalChannelId](#)
 - Parameter [I2sTxCallback](#)
 - Parameter [I2sRxCallback](#)
 - Reference [I2sModuleRef](#)
 - Reference [I2sChannelEcucPartitionRef](#)
 - * Container [SaiHwConfiguration](#)
 - Parameter [SaiInstanceId](#)
 - Parameter [SaiTxRxConfiguration](#)
 - Parameter [SaiMode](#)
 - Parameter [SaiMasterClkSrc](#)
 - Parameter [SaiSyncType](#)
 - Parameter [SaiBitClkSwap](#)

- Parameter [SaiBitClkNegPolar](#)
- Parameter [SaiSampleRate](#)
- Parameter [SaiDataLine0](#)
- Parameter [SaiDataLine1](#)
- Parameter [SaiDataLine2](#)
- Parameter [SaiDataLine3](#)
- Parameter [SaiFrameSize](#)
- Parameter [SaiSyncWidth](#)
- Parameter [SaiChannelMode](#)
- Parameter [SaiMsbFirst](#)
- Parameter [SaiEnableDebug](#)
- Parameter [SaiSyncEarly](#)
- Parameter [SaiSyncNegPolar](#)
- Parameter [SaiWord0Width](#)
- Parameter [SaiWordNWidth](#)
- Parameter [SaiFirstBitIndex](#)
- Parameter [SaiRunErrorReport](#)
- Parameter [SaiSyncErrorReport](#)
- Parameter [SaiFrameStartReport](#)
- Parameter [SaiFifoPackEnable](#)
- Parameter [SaiMuxMode](#)
- Parameter [SaiTransferType](#)
- Parameter [SaiElementSize](#)
- Parameter [SaiBufferAccessSize](#)
- Parameter [SaiDataLineCount](#)
- Parameter [SaiWordFlagIndex](#)
- Parameter [SaiBitClkInput](#)
- Parameter [SaiContOnErr](#)
- Parameter [SaiSyncOnDemand](#)
- Reference [SaiClkSrcRef](#)
- Reference [SaiDmaChannel0](#)
- Reference [SaiDmaChannel1](#)
- Reference [SaiDmaChannel2](#)
- Reference [SaiDmaChannel3](#)
- * Container [FlexioHwConfiguration](#)
 - Parameter [FlexioI2sInstanceId](#)

- Parameter [FlexioI2sBitsWidth](#)
- Parameter [FlexioI2sMode](#)
- Parameter [FlexioI2sBaudRate](#)
- Parameter [FlexioI2sTransferType](#)
- Parameter [FlexioI2sTxLine](#)
- Parameter [FlexioI2sRxLine](#)
- Parameter [FlexioI2sWsLine](#)
- Parameter [FlexioI2sClkSamplingEdge](#)
- Parameter [FlexioI2sTxCallbackParam](#)
- Parameter [FlexioI2sRxCallbackParam](#)
- Reference [FlexioI2sClkSrcRef](#)
- Reference [FlexioI2sDataLine](#)
- Reference [FlexioI2sSckLine](#)
- Reference [FlexioI2sDmaTxChannel](#)
- Reference [FlexioI2sDmaRxChannel](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)
- Container [I2sClockReferencePoint](#)
 - * Reference [I2sClockReference](#)

4.1.1 Module I2s

Configuration of the I2s module.

Included containers:

- [GeneralConfiguration](#)
 - [I2sConfigSet](#)
 - [CommonPublishedInformation](#)
 - [I2sClockReferencePoint](#)
-

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar I2s driver.

Included subcontainers:

- [I2sSaiSupport](#)
- [I2sFlexioSupport](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter I2sDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.4 Parameter I2sMultipartitionSupport

This parameter globally enables the possibility to support multipartition. If I2sMultipartitionSupport is disabled, then for all the variants no partition shall be defined. If I2sMultipartitionSupport is enabled, at least one EcucPartition needs to be defined (in all

variants).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.5 Parameter I2sEnableUserModeSupport

I2s Enable User Mode Support

When this parameter is enabled, the I2s module will adapt to run from User Mode, with the following measure : configuring REG_PROT for I2s IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1 for more information and availability on this platform, please see chapter "User Mode Support" in IM.

True: I2s module will adapt to run from User Mode.

False:I2s module will not adapt to run from User Mode.

If this parameter is not ediatable, that means I2s driver can run in User Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.6 Parameter I2sVersionInfoApi

Adds / removes the service I2s_GetVersionInfo() from the code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.7 Parameter I2sTimeoutMethod

Configures the timeout method for I2s.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.1.8 Parameter I2sTimeoutVal

This is a timeout (microseconds) value which is used to wait for each synchronization transfer

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	4294967295
min	1

4.1.9 Container I2sSaiSupport

Vendor specific: Container for the I2s Sai related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.10 Parameter I2sSaiEnable

Vendor specific: Enable/Disable Sai support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.11 Parameter SaiIpDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.12 Parameter SaiDmaFeature

Enable DMA feature for transferring and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.13 Parameter SaiAutoDisableClockFeature

Sai Auto Disable Clock Feature. When enabled, allows the driver to disable clock generation after transmission completed or aborted. The clock generation will be started again at next transmission.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.14 Parameter SaiFifoPackFeature

Enable FIFO packing feature. When this parameter is enabled, the Sai FIFO Packing Enable can be configured.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.15 Parameter SaiBitClkSwapSupport

Sai Enable Bit Clock Swap Support

When this parameter is enabled, the Sai Bit Clk Swap can be configured.

If this parameter is not ediatable, that means this feature is not available on this device.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.16 Parameter SaiEnableDataMasking

Enable to allow masking of sent or received data. Data will be masked according to the SaiChannelMode selection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.17 Container I2sFlexioSupport

Vendor specific: Container for the I2s Flexio related configuration parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.18 Parameter I2sFlexioI2sEnable

Vendor specific: Enable/Disable Flexio support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	true

4.1.19 Parameter FlexioI2sIpDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.20 Parameter FlexioI2sDmaFeature

Enable DMA feature for transferring and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.21 Container I2sConfigSet

This container contains the configuration parameters and sub containers of the I2s module.

Included subcontainers:

- [I2sConfiguration](#)
- [SaiHwConfiguration](#)
- [FlexioHwConfiguration](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.22 Container I2sConfiguration

This container contains the configuration (parameters) of the I2s Controller(s).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.23 Parameter I2sIpType

Selects Hardware IP.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	SAI
literals	['SAI', 'FLEXIO_I2S']

4.1.24 Parameter I2sLogicalChannelId

Identifies the I2s Logical channel ID partitionsponding with I2s configuration.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Configuration Parameters

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	0

4.1.25 Parameter I2sTxCallback

I2s Tx callback. This function will be called for all I2s Tx events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

4.1.26 Parameter I2sRxCallback

I2s Rx callback. This function will be called for all I2s Rx events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

4.1.27 Reference I2sModuleRef

Maps an I2s channel to zero or one ECUC partition to limit the access to this channel group. The ECUC partitions referenced are a subset of the ECUC partitions where the I2s driver is mapped to.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/I2s/I2sConfigSet/SaiHwConfiguration', '/TS_T40D2M30I0R0/I2s/I2sConfigSet/FlexioHwConfiguration']

4.1.28 Reference I2sChannelEcucPartitionRef

Maps a I2s channel to zero or one ECUC partition to limit the access to this channel group. The ECUC partitions referenced are a subset of the ECUC partitions where the I2s driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.29 Container SaiHwConfiguration

Configuration of a Sai module available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	0
upperMultiplicity	4
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.30 Parameter SaiInstanceId

Identifies the Sai Instance.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	0
min	0

4.1.31 Parameter SaiTxRxConfiguration

Selects Tx or Rx configuration.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Transmitter
literals	['Transmitter', 'Receiver']

4.1.32 Parameter SaiMode

Master or Slave.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	Master
literals	['Master', 'Slave']

4.1.33 Parameter SaiMasterClkSrc

Select master clock source.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	UNSUPPORTED
literals	['UNSUPPORTED']

4.1.34 Parameter SaiSyncType

SAI run in sync or async mode.

ASYNCR: Independent clock.

SYNC_WITH_OTHER: Bit clock and frame sync signal is taken from transmitter/receiver.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	ASYNC
literals	['ASYNC', 'SYNC_WITH_OTHER']

4.1.35 Parameter SaiBitClkSwap

This field swaps the bit clock used by the transmitter. When the transmitter is configured in asynchronous mode and this bit is set, the transmitter is clocked by the receiver bit clock (RX_BCLK). This allows the transmitter and receiver to share the same bit clock, but the transmitter continues to use the transmit frame sync (TX_SYNC).

When the transmitter is configured in synchronous mode, the transmitter BCS field and receiver BCS field must be set to the same value. When both are set, the transmitter and receiver are both clocked by the transmitter bit clock (TX_BCLK) but use the receiver frame sync (RX_SYNC).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.36 Parameter SaiBitClkNegPolar

True if bit clock is negative polar (active low with drive outputs on falling edge and sample inputs on rising edge), false otherwise.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.37 Parameter SaiSampleRate

Sample rate for the Sai module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	16000
max	4294967295
min	1

4.1.38 Parameter SaiDataLine0

Enable or disable Sai Data Line 0.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.39 Parameter SaiDataLine1

Enable or disable Sai Data Line 1.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.40 Parameter SaiDataLine2

Enable or disable Sai Data Line 2.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.41 Parameter SaiDataLine3

Enable or disable Sai Data Line 3.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.42 Parameter SaiFrameSize

Frame size in number of words. When in mux line mode, the number of unmasked word in a frame must be divisible by number of channels.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	16
min	1

4.1.43 Parameter SaiSyncWidth

Active sync width in number of bit clocks (must not longer than first word width).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	32
min	1

4.1.44 Parameter SaiChannelMode

Configures if transmit data pins are configured for TDM mode or Output mode.

MASK_TRISTATE - TDM mode, transmit data pins are tri-stated when slots are masked or channels are disabled.

MASK_ZERO - Output mode, transmit data pins are never tri-stated and will output zero when slots are masked or channels are disabled.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	MASK_TRISTATE
literals	['MASK_TRISTATE', 'MASK_ZERO']

4.1.45 Parameter SaiMsbFirst

True if data is MSB first, false if LSB first.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	MSB
literals	['MSB', 'LSB']

4.1.46 Parameter SaiEnableDebug

Enables/disables transmitter or receiver operation in Debug mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.47 Parameter SaiSyncEarly

True if frame sync is one bit clock early.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.48 Parameter SaiSyncNegPolar

True if frame sync is negative polarity (active low), false otherwise.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.49 Parameter SaiWord0Width

Configures the number of bits in the first word in each frame.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	16
max	32
min	1

4.1.50 Parameter SaiWordNWidth

Configures the number of bits in each word except the first word in the frame.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	16
max	32
min	8

4.1.51 Parameter SaiFirstBitIndex

Index from LSB of first bit to be transmitted/received, valid range from 0 to 31.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	31
min	0

4.1.52 Parameter SaiRunErrorReport

Execute the callback function on underrun/overflow event.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.53 Parameter SaiSyncErrorReport

Execute the callback function when an error on external frame sync has occurred.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.54 Parameter SaiFrameStartReport

Execute the callback function on frame start event event.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.55 Parameter SaiFifoPackEnable

Sai FIFO Packing Enable

Enable FIFO packing base on element size:

1-byte: 8-bit FIFO packing is enabled.

2-byte: 16-bit FIFO packing is enabled.

4-byte: FIFO packing is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.56 Parameter SaiMuxMode

Mux type.

MUX_DISABLED: Each data line is a channel, uses a seperate memory block.

MUX_LINE: Only one data line (data line 0 for DMA mode) is used. Words on data line is alternated between channels, each channel data is a seperate memory block.

MUX_MEM: Words in memory block is alternated between channels, each channel data is on a seperate data line.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MUX_DISABLED
literals	['MUX_DISABLED', 'MUX_LINE', 'MUX_MEM']

4.1.57 Parameter SaiTransferType

Select the Sai transfer type, using Interrupt or DMA.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	INTERRUPT
literals	['INTERRUPT', 'DMA']

4.1.58 Parameter SaiElementSize

Size in bytes of each read/write from/to buffer and should not be less than Sai Word Width.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	2
max	4
min	1

4.1.59 Parameter SaiBufferAccessSize

Number of read/write operations done on FIFO during one interrupt/dma operation. Maximum value is given by FIFO depth.

When FIFO packing is disabled, this value is equal with number of words each time read from/write to buffer.

If FIFO packing is enabled, the driver will actually read from/write to buffer the number multiplied by number of words packed into 1 FIFO read/write operation(example: N, with 1byte words and packing => $N \cdot (32/8)$ elements read from buffer per operation).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4
max	8
min	1

4.1.60 Parameter SaiDataLineCount

Number of Sai Data Lines to enable, only used when both line mux mode and interrupt mode is selected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
defaultValue	2
max	4
min	2

4.1.61 Parameter SaiWordFlagIndex

Configure which word sets the start of word flag, should less than frame size.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.1.62 Parameter SaiBitClkInput

Use slave mode timing in datasheet for master mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.63 Parameter SaiContOnErr

True continue transferring the same work that caused FIFO error, False start the next frame.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.64 Parameter SaiSyncOnDemand

True frame sync is generated only when fifo is not empty (transmit) or not full (receive), False otherwise.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.65 Reference SaiClkSrcRef

Reference to the I2sClockReferencePoint from which the clock is derived.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False

Property	Value
destination	/TS_T40D2M30I0R0/I2s/I2sClockReferencePoint

4.1.66 Reference SaiDmaChannel0

Specifies channel 0 DMA channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.67 Reference SaiDmaChannel1

Specifies channel 1 DMA channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.68 Reference SaiDmaChannel2

Specifies channel 2 DMA channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF

Configuration Parameters

Property	Value
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.69 Reference SaiDmaChannel3

Specifies channel 3 DMA channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.70 Container FlexioHwConfiguration

Configuration of a Flexio module available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.71 Parameter FlexioI2sInstanceId

Identifies the Flexio Channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	3
min	0

4.1.72 Parameter FlexioI2sBitsWidth

Number of bytes in a word - Support only 1 byte,2 byte,4 byte.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4
min	1

4.1.73 Parameter FlexioI2sMode

Master or Slave.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	Master
literals	['Master', 'Slave']

4.1.74 Parameter FlexioI2sBaudRate

The actual baud rate in Hz that can be set by the driver. May be substantially different than the requested baud rate if the frequency of the Flexio input clock is too high or too low.

For best results it is recommended to use an input clock with a frequency 200-500 times greater than the desired baud rate

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	20000
max	4294967295
min	1

4.1.75 Parameter FlexioI2sTransferType

Select the Flexio I2s transfer type, using Interrupt or Polling or DMA.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
default Value	INTERRUPT
literals	['INTERRUPT', 'POLLING', 'DMA']

4.1.76 Parameter FlexioI2sTxLine

FlexioI2sTxLine - ENABLED/DISABLED

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.77 Parameter FlexioI2sRxLine

Enable 'FlexioI2sRxLine' - If both TX and RX lines are enabled please ENABLE Flexio Add pin in the referenced Mcl Flexio Configuration from FlexioI2sDataLine. .

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.78 Parameter FlexioI2sWsLine

FlexioI2sWsLine is enabled indirectly via the label node 'Enable Flexio Add pin' of Flexio_Mcl_Ip component referenced from FlexioI2sSckLine.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.79 Parameter FlexioI2sClkSamplingEdge

Configure the positive edge of data sampling clk.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.80 Parameter FlexioI2sTxCallbackParam

Flexio I2s Txcallback Param. This param will be called for all Flexio I2s Tx events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
defaultValue	NULL_PTR

4.1.81 Parameter FlexioI2sRxCallbackParam

Flexio I2s Rx callback Param. This param will be called for all Flexio I2s Rx events.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

4.1.82 Reference FlexioI2sClkSrcRef

Reference to the I2sClockReferencePoint from which the clock is derived.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/I2s/I2sClockReferencePoint

4.1.83 Reference FlexioI2sDataLine

Reference to the FLEXIO logic channel, which is set in the Mcl driver configuration.

In Master mode, Channel N is referred in Data line and Channel N+1 is referred in SCK line.

In Slave mode, Channel N is referred in Data line and Channel N+1/N+2 are referred in SCK line.

Note: Implementation Specific Parameter.

Configuration Parameters

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels

4.1.84 Reference FlexioI2sSckLine

Reference to the FLEXIO logic channel, which is set in the Mcl driver configuration.

In Master mode, Channel N is referred in TX line and Channel N+1 is referred in SCK line.

In Slave mode, Channel N is referred in TX line and Channel N+1/N+2 are referred in SCK line.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D2M30I0R0/Mcl/MclConfig/FlexioCommon/FlexioMclLogicChannels

4.1.85 Reference FlexioI2sDmaTxChannel

Specifies Flexio I2s Tx Dma Channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.86 Reference FlexioI2sDmaRxChannel

Specifies Flexio I2s Rx Dma Channel used.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.87 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.88 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.89 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.90 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.91 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.1.92 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.93 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.94 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.95 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.96 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.1.97 Container I2sClockReferencePoint

This container contains a parameter, which represents a reference to a container of the type McuClockReferencePoint (defined in module MCU).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.98 Reference I2sClockReference

Reference to a container of the type *McuClockReferencePoint*, to select an input clock.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.2 SAI_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Sai_Ip](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [SaiIpDevErrorDetect](#)
 - * Parameter [SaiEnableUserModeSupport](#)
 - * Parameter [SaiDmaFeature](#)
 - * Parameter [SaiAutoDisableClockFeature](#)
 - * Parameter [SaiFifoPackFeature](#)
 - * Parameter [SaiBitClkSwapSupport](#)
 - * Parameter [SaiEnableDataMasking](#)
 - * Parameter [SaiTimeoutMethod](#)
 - * Parameter [SaiTimeoutVal](#)
 - Container [SaiHwConfiguration](#)

4.2.1 Module Sai_Ip

Configuration of the *Sai_Ip* module

Included containers:

- [GeneralConfiguration](#)
- [SaiHwConfiguration](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.2.2 Parameter SaiEnableUserModeSupport

Sai Enable User Mode Support

When this parameter is enabled, the Sai module will adapt to run from User Mode, with the following measure : configuring REG_PROT for Sai IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1 for more information and availability on this platform, please see chapter "User Mode Support" in IM.

True: Sai module will adapt to run from User Mode.

False:Sai module will not adapt to run from User Mode.

If this parameter is not ediatable, that means Sai driver can run in User Mode.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.2.3 Parameter SaiTimeoutMethod

Configures the timeout method for I2s.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Property	Value
type	ENUMERATION
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	OSIF_COUNTER_DUMMY
literals	[]

4.2.4 Parameter SaiTimeoutVal

This is a timeout (microseconds) value which is used to wait for each synchronization transfer

Property	Value
type	INTEGER
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1000
max	4294967295
min	1

4.3 FLEXIO_I2S_IP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Flexio_I2s_Ip](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [FlexioI2sIpDevErrorDetect](#)

- * Parameter [FlexioI2sEnableUserModeSupport](#)
- * Parameter [FlexioI2sDmaFeature](#)
- * Parameter [FlexioI2sTimeoutMethod](#)
- * Parameter [FlexioI2sTimeoutVal](#)
- Container [FlexioHwConfiguration](#)

4.3.1 Module Flexio_I2s_Ip

Configuration of the Flexio_I2s_Ip module

Included containers:

- [GeneralConfiguration](#)
- [FlexioHwConfiguration](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3.2 Parameter FlexioI2sEnableUserModeSupport

Flexio I2s Enable User Mode Support

When this parameter is enabled, the Flexio I2s module will adapt to run from User Mode, with the following measure : configuring REG_PROT for Flexio I2s IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1 for more information and availability on this platform, please see chapter "User Mode Support" in IM.

True: Flexio I2s module will adapt to run from User Mode.

False: Flexio I2s module will not adapt to run from User Mode.

If this parameter is not ediatable, that means Flexio I2s driver can run in User Mode.

Property	Value
type	BOOLEAN
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.3.3 Parameter FlexioI2sTimeoutMethod

Configures the timeout method for I2s.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Property	Value
type	ENUMERATION
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	OSIF_COUNTER_DUMMY
literals	[]

4.3.4 Parameter FlexioI2sTimeoutVal

This is a timeout (microseconds) value which is used to wait for each synchronization transfer

Property	Value
type	INTEGER
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	4294967295
min	1



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

I2s Driver	63
FLEXIO_I2S IPL	75
SAI IPL	84

Chapter 6

Module Documentation

6.1 I2s Driver

6.1.1 Detailed Description

Data Structures

- struct [I2s_StatusStructType](#)
Definition of the type of transfer status. [More...](#)
- struct [I2s_ConfigType](#)
This type contains initialization data. [More...](#)
- struct [I2s_RequestType](#)
Definition for Request Buffer. This is the structure which is passed to `I2s_SyncTransmit` or `I2s_AsyncTransmit` function. This holds the necessary information required for the communication of I2S Hw with the Slave device. [More...](#)
- struct [I2s_Ipw_ConfigType](#)
This structure contains config structures for all IPs available. [More...](#)
- struct [I2s_HwConfigType](#)
This structure holds the HWUnit configuration parameters. [More...](#)
- struct [I2s_LogicalConfigState](#)
This structure holds the Logical configuration parameters. [More...](#)

Macros

- `#define I2S_E_UNINIT`
API service used without I2s module initialization.
- `#define I2S_E_BUSY`
I2s module is busy with a running operation.
- `#define I2S_E_TIMEOUT`
I2s module is timeout with a running operation.
- `#define I2S_E_INIT_FAILED`
I2s module is not properly initialized.
- `#define I2S_E_PARAM_POINTER`
API service is called using an invalid pointer (e.g. the pointer should not be NULL).

- `#define I2S_E_PARAM_CHANNEL`
API service is called using an invalid logical channel.
- `#define I2S_E_PARAM_CONFIG`
API service is called using an invalid partition.
- `#define I2S_E_INCORRECT_DIRECTION`
I2s module is not properly direction initialized.
- `#define I2S_INIT_ID`
API service ID for I2s_Init function.
- `#define I2S_DEINIT_ID`
API service ID for I2s_DeInit function.
- `#define I2S_SYNCTRANSMIT_ID`
API service ID for I2s_SyncTransmit function.
- `#define I2S_ASYNCTRANSMIT_ID`
API service ID for I2s_AsyncTransmit function.
- `#define I2S_GETSTATUS_ID`
API service ID for I2s_GetStatus function.
- `#define I2S_ABORTTRANSMIT_ID`
API service ID for I2s_AbortTransmit function.
- `#define I2S_GETVERSIONINFO_ID`
API service ID for I2s_GetVersionInfo function.

Types Reference

- `typedef uint8 I2s_HwUnitCount`
This gives the number of I2S configuration.
- `typedef uint8 I2s_DataType`
This gives the transfer data type.
- `typedef uint16 I2s_WordMaskType`
I2s Word Mask for transmitted data.
- `typedef uint8 I2s_LogicalChannel`
This gives the logical channel.

Enum Reference

- `enum I2s_DataDirectionType`
Definition of the type of transfer direction.
- `enum I2s_StatusType`
Definition for different state and errors of Operation Status.
- `enum I2s_TransmitReceiveType`
Definition of the transmitter/receiver of a SAI hw unit.
- `enum I2s_ModeType`
Definition of the master/slave mode.
- `enum I2s_SupportedIPsType`

This enum contains all IPs which can integrate SAI functionalities.

Function Reference

- void **I2s_Init** (const **I2s_ConfigType** *const pConfigPtr)
Initializes the I2S module.
- void **I2s_DeInit** (void)
DeInitializes the I2S module.
- Std_ReturnType **I2s_SyncTransmit** (**I2s_LogicalChannel** LogicChn, const **I2s_RequestType** *const pRequest)
Sends or receives an I2S message blocking.
- Std_ReturnType **I2s_AsyncTransmit** (**I2s_LogicalChannel** LogicChn, const **I2s_RequestType** *const pRequest)
Starts an asynchronous transmission on the I2S bus.
- Std_ReturnType **I2s_GetStatus** (**I2s_LogicalChannel** LogicChn, **I2s_StatusStructType** *const pStatus)
Gets the status of an I2S instance.
- void **I2s_AbortTransmit** (**I2s_LogicalChannel** LogicChn)
Aborts the I2s ongoing transmission.
- void **I2s_GetVersionInfo** (Std_VersionInfoType *pVersionInfo)
Returns the version information of this module.

6.1.2 Data Structure Documentation

6.1.2.1 struct I2s_StatusStructType

Definition of the type of transfer status.

Definition at line 146 of file CDD_I2s_Types.h.

6.1.2.2 struct I2s_ConfigType

This type contains initialization data.

This contains initialization data for the I2S driver. It shall contain:

- The number of I2S modules to be configured
- I2S dependent properties for used HW units

Definition at line 167 of file CDD_I2s_Types.h.

Data Fields

const I2s_HwUnitCount	cfgCount	Number of I2S configurations. < Pointer to I2s hardware unit configuration array
const I2s_HwConfigType *const *	pHwConfig	

6.1.2.3 struct I2s_RequestType

Definition for Request Buffer. This is the structure which is passed to **I2s_SyncTransmit** or **I2s_AsyncTransmit** function. This holds the necessary information required for the communication of I2S Hw with the Slave device.

Definition at line 181 of file CDD_I2s_Types.h.

Data Fields

uint32	bufferSize	Buffer Size : The number of words for reading or writing of each channel. The size of each buffer word element is "I2s Buffer Element Size" selected in configurator. < Direction of the data. Can be either Send or Receive.
I2s_DataDirectionType	dataDirection	
boolean	autoDisableClock	Auto disable clock when the request is completed (If hardware supports) (Saving power if there is no request immediately after) The clock generation will be started again at next transmission. <
I2s_DataType **	pDataBuffer	Buffer to Store or to transmit Serial data. <

6.1.2.4 struct I2s_Ipw_ConfigType

This structure contains config structures for all IPs available.

Definition at line 193 of file I2s_Ipw_Types.h.

6.1.2.5 struct I2s_HwConfigType

This structure holds the HWUnit configuration parameters.

Definition at line 208 of file I2s_Ipw_Types.h.

6.1.2.6 struct I2s_LogicalConfigState

This structure holds the Logical configuration parameters.

Definition at line 223 of file I2s_Ipw_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 I2S_E_UNINIT

```
#define I2S_E_UNINIT
```

API service used without I2s module initialization.

Development errors. The following errors shall be detectable by the I2S module depending on its configuration (development / production mode).

All error codes

Definition at line 135 of file CDD_I2s.h.

6.1.3.2 I2S_E_BUSY

```
#define I2S_E_BUSY
```

I2s module is busy with a running operation.

Definition at line 140 of file CDD_I2s.h.

6.1.3.3 I2S_E_TIMEOUT

```
#define I2S_E_TIMEOUT
```

I2s module is timeout with a running operation.

Definition at line 145 of file CDD_I2s.h.

6.1.3.4 I2S_E_INIT_FAILED

```
#define I2S_E_INIT_FAILED
```

I2s module is not properly initialized.

Definition at line 150 of file CDD_I2s.h.

6.1.3.5 I2S_E_PARAM_POINTER

```
#define I2S_E_PARAM_POINTER
```

API service is called using an invalid pointer (e.g. the pointer should not be NULL).

Definition at line 155 of file CDD_I2s.h.

6.1.3.6 I2S_E_PARAM_CHANNEL

```
#define I2S_E_PARAM_CHANNEL
```

API service is called using an invalid logical channel.

Definition at line 160 of file CDD_I2s.h.

6.1.3.7 I2S_E_PARAM_CONFIG

```
#define I2S_E_PARAM_CONFIG
```

API service is called using an invalid partition.

Definition at line 165 of file CDD_I2s.h.

6.1.3.8 I2S_E_INCORRECT_DIRECTION

```
#define I2S_E_INCORRECT_DIRECTION
```

I2s module is not properly direction initialized.

Definition at line 170 of file CDD_I2s.h.

6.1.3.9 I2S_INIT_ID

```
#define I2S_INIT_ID
```

API service ID for I2s_Init function.

All AUTOSAR API's service IDs

Definition at line 179 of file CDD_I2s.h.

6.1.3.10 I2S_DEINIT_ID

```
#define I2S_DEINIT_ID
```

API service ID for I2s_DeInit function.

Definition at line 184 of file CDD_I2s.h.

6.1.3.11 I2S_SYNCTRANSMIT_ID

```
#define I2S_SYNCTRANSMIT_ID
```

API service ID for I2s_SyncTransmit function.

Definition at line 189 of file CDD_I2s.h.

6.1.3.12 I2S_ASYNCTRANSMIT_ID

```
#define I2S_ASYNCTRANSMIT_ID
```

API service ID for I2s_AsyncTransmit function.

Definition at line 194 of file CDD_I2s.h.

6.1.3.13 I2S_GETSTATUS_ID

```
#define I2S_GETSTATUS_ID
```

API service ID for I2s_GetStatus function.

Definition at line 199 of file CDD_I2s.h.

6.1.3.14 I2S_ABORTTRANSMIT_ID

```
#define I2S_ABORTTRANSMIT_ID
```

API service ID for I2s_AbortTransmit function.

Definition at line 204 of file CDD_I2s.h.

6.1.3.15 I2S_GETVERSIONINFO_ID

```
#define I2S_GETVERSIONINFO_ID
```

API service ID for I2s_GetVersionInfo function.

Definition at line 209 of file CDD_I2s.h.

6.1.4 Types Reference

6.1.4.1 I2s_HwUnitCount

```
typedef uint8 I2s_HwUnitCount
```

This gives the number of I2S configuration.

Definition at line 126 of file CDD_I2s_Types.h.

6.1.4.2 I2s_DataType

```
typedef uint8 I2s_DataType
```

This gives the transfer data type.

Definition at line 131 of file CDD_I2s_Types.h.

6.1.4.3 I2s_WordMaskType

```
typedef uint16 I2s_WordMaskType
```

I2s Word Mask for transmitted data.

Definition at line 136 of file CDD_I2s_Types.h.

6.1.4.4 I2s_LogicalChannel

```
typedef uint8 I2s_LogicalChannel
```

This gives the logical channel.

Definition at line 141 of file CDD_I2s_Types.h.

6.1.5 Enum Reference

6.1.5.1 I2s_DataDirectionType

```
enum I2s_DataDirectionType
```

Definition of the type of transfer direction.

Enumerator

I2S_SEND_DATA	Used to send data.
I2S_RECEIVE_DATA	Used to receive data.

Definition at line 155 of file CDD_I2s_Types.h.

6.1.5.2 I2s__StatusType

enum `I2s__StatusType`

Definition for different state and errors of Operation Status.

Enumerator

I2S__STATUS_UNINITIALIZED	Function unsupported
I2S__STATUS_COMPLETED	Function completed successfully
I2S__STATUS_ERROR	Function didn't complete successfully
I2S__STATUS_TIMEOUT	Function timed out
I2S__STATUS_BUSY	Function busy
I2S__STATUS_ABORTED	Function aborted

Definition at line 149 of file I2s_Ipw_Types.h.

6.1.5.3 I2s__TransmitReceiveType

enum `I2s__TransmitReceiveType`

Definition of the transmitter/receiver of a SAI hw unit.

Enumerator

I2S__TRANSMITTER	Transmitter.
I2S__RECEIVER	Receiver.

Definition at line 162 of file I2s_Ipw_Types.h.

6.1.5.4 I2s__ModeType

enum `I2s__ModeType`

Definition of the master/slave mode.

Enumerator

I2S__MASTER	Master.
I2S__SLAVE	Slave.

Definition at line 171 of file I2s_Ipw_Types.h.

6.1.5.5 I2s_SupportedIPsType

enum [I2s_SupportedIPsType](#)

This enum contains all IPs which can integrate SAI functionalities.

Definition at line 180 of file I2s_Ipw_Types.h.

6.1.6 Function Reference

6.1.6.1 I2s_Init()

```
void I2s_Init (
    const I2s\_ConfigType *const pConfigPtr )
```

Initializes the I2S module.

This function performs software initialization of I2S driver.

Parameters

in	<i>pConfig</i>	Pointer to I2S driver configuration set.
----	----------------	--

Returns

void

Note

Service ID: 0x00.

Synchronous, non re-entrant function.

6.1.6.2 I2s_DeInit()

```
void I2s_DeInit (
    void )
```

DeInitializes the I2S module.

This function performs software de initialization of I2S modules to reset values. The service influences only the peripherals, which are allocated by static configuration and the runtime configuration set passed by the previous call of [I2s_Init\(\)](#). The driver needs to be initialized before calling [I2s_DeInit\(\)](#). Otherwise, the function [I2s_DeInit](#) shall raise the development error `I2S_E_UNINIT` and leave the desired de initialization functionality without any action.

Parameters

in	<i>void</i>	
----	-------------	--

Returns

void

Note

Service ID: 0x01.

Synchronous, non re-entrant function.

6.1.6.3 I2s_SyncTransmit()

```
Std_ReturnType I2s_SyncTransmit (
    I2s_LogicalChannel LogicChn,
    const I2s_RequestType *const pRequest )
```

Sends or receives an I2S message blocking.

Sends the slave address and based on the direction of the message it sends or receives data by using a blocking mechanism.

Parameters

in	<i>LogicChn</i>	I2S logical channel to be addressed.
in	<i>pRequest</i>	Pointer to data information to be used

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2S Instance is not valid or I2S driver is not initialized or pRequest is NULL or I2S Instance is in busy state.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x02.

Synchronous, non reentrant function.

6.1.6.4 I2s_AsyncTransmit()

```
Std_ReturnType I2s_AsyncTransmit (
    I2s_LogicalChannel LogicChn,
    const I2s_RequestType *const pRequest )
```

Starts an asynchronous transmission on the I2S bus.

Sends the slave address and enables the interrupts that will send or receive data depending on the direction of the message.

Parameters

in	<i>LogicChn</i>	I2S logical channel to be addressed.
in	<i>pRequest</i>	Pointer to data information to be used

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2S Instance is not valid or I2S driver is not initialized or pRequest is NULL or I2S Instance is in busy state.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x03.

Synchronous, non reentrant function.

6.1.6.5 I2s_GetStatus()

```
Std_ReturnType I2s_GetStatus (
    I2s_LogicalChannel LogicChn,
    I2s_StatusStructType *const pStatus )
```

Gets the status of an I2S instance.

Gets the status of an I2S instance and checks for errors.

Parameters

in	<i>LogicChn</i>	I2S logical channel to be addressed.
out	<i>pStatus</i>	Pointer for storing the current transfer status

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2S Instance is not valid or I2S driver is not initialized or pStatus is NULL.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x04.

Synchronous, non re-entrant function.

6.1.6.6 I2s_AbortTransmit()

```
void I2s_AbortTransmit (
    I2s_LogicalChannel LogicChn )
```

Aborts the I2s ongoing transmission.

Aborts the I2s ongoing transmission.

Parameters

in	<i>LogicChn</i>	I2S logical channel to be addressed.
----	-----------------	--------------------------------------

Returns

void.

Note

Service ID: 0x05.

Synchronous, non reentrant function.

6.1.6.7 I2s_GetVersionInfo()

```
void I2s_GetVersionInfo (
    Std_VersionInfoType * pVersionInfo )
```

Returns the version information of this module.

The version information includes:

- Two bytes for the Vendor ID
- Two bytes for the Module ID
- One byte for the Instance ID
- Three bytes version number. The numbering shall be vendor specific: it consists of:
 - The major, the minor and the patch version number of the module;
 - The AUTOSAR specification version number shall not be included. The AUTOSAR specification version number is checked during compile time and therefore not required in this API.

Parameters

out	<i>pVersionInfo</i>	Pointer for storing the version information of this module.
-----	---------------------	---

Returns

void.

Precondition

Preconditions as text description. Optional tag.

Note

Service ID: 0x0A.

Synchronous, non re-entrant function.

6.2 FLEXIO_I2S IPL

6.2.1 Detailed Description

Data Structures

- struct [Flexio_I2s_Ip_CommonStateType](#)
- struct [Flexio_I2s_Ip_StateType](#)
Master internal context structure. [More...](#)
- struct [Flexio_I2s_Ip_MasterConfigType](#)
Master configuration structure. [More...](#)
- struct [Flexio_I2s_Ip_SlaveConfigType](#)
Slave configuration structure. [More...](#)

Enum Reference

- enum [Flexio_I2s_Ip_StatusType](#)
Status type Implements : [Flexio_I2s_Ip_StatusType_Class](#).
- enum [Flexio_I2s_Ip_DriverType](#)
Driver type: INTERRUPT/POLLING/DMA Implements : [Flexio_I2s_Ip_DriverType_Class](#).
- enum [Flexio_I2s_Ip_EventType](#)

Function Reference

- void [Flexio_I2s_Ip_MasterInit](#) (uint8 Instance, uint8 Channel, const [Flexio_I2s_Ip_MasterConfigType](#) *ConfigPtr)
Initialize the FLEXIO_I2S master mode driver.
- void [Flexio_I2s_Ip_Deinit](#) (uint8 Instance, uint8 Channel)
De-initialize the FLEXIO_I2S driver.
- [Flexio_I2s_Ip_StatusType](#) [Flexio_I2s_Ip_SendData](#) (uint8 Instance, uint8 Channel, const uint8 *TxBuff, uint32 TxSize)
Perform a non-blocking send transaction on the I2S bus.
- [Flexio_I2s_Ip_StatusType](#) [Flexio_I2s_Ip_SendDataBlocking](#) (uint8 Instance, uint8 Channel, const uint8 *TxBuff, uint32 TxSize, uint32 Timeout)
Perform a blocking send transaction on the I2S bus.
- [Flexio_I2s_Ip_StatusType](#) [Flexio_I2s_Ip_ReceiveData](#) (uint8 Instance, uint8 Channel, uint8 *RxBuff, uint32 RxSize)
Perform a non-blocking receive transaction on the I2S bus.
- [Flexio_I2s_Ip_StatusType](#) [Flexio_I2s_Ip_ReceiveDataBlocking](#) (uint8 Instance, uint8 Channel, uint8 *RxBuff, uint32 RxSize, uint32 Timeout)
Perform a blocking receive transaction on the I2S bus.
- [Flexio_I2s_Ip_StatusType](#) [Flexio_I2s_Ip_TransferAbort](#) (uint8 Instance, uint8 Channel)
Aborts a non-blocking I2S transaction.
- [Flexio_I2s_Ip_StatusType](#) [Flexio_I2s_Ip_GetStatus](#) (uint8 Instance, uint8 Channel, uint32 *BytesRemaining)
Get the status of the current non-blocking I2S transaction.

- void [Flexio_I2s_Ip_SlaveInit](#) (uint8 Instance, uint8 Channel, const [Flexio_I2s_Ip_SlaveConfigType](#) *ConfigPtr)

Initialize the FLEXIO_I2S slave mode driver.

- void [Flexio_I2s_Ip_IrqHandler](#) (uint8 FlexIOChannel, uint8 ShifterMaskFlag, uint8 ShifterErrMaskFlag, uint8 TimerMaskFlag)

Interrupt handler for FlexIO.

6.2.2 Data Structure Documentation

6.2.2.1 struct Flexio_I2s_Ip_CommonStateType

FlexIO common context structure This is a common structure used by all FlexIO drivers as a part of their context structure. It is needed for common operations such as interrupt handling.

Definition at line 137 of file [Flexio_I2s_Ip_Types.h](#).

Data Fields

uint8	Instance	FlexIO device instance number
uint8	ResourceCount	Count of internal resources used (shifters and timers)
uint8	ResourceIndex	Index of first used internal resource instance (shifter and timer)
boolean	TxEnabled	
boolean	RxEnabled	
uint8	TxShifter	
uint8	RxShifter	

6.2.2.2 struct Flexio_I2s_Ip_StateType

Master internal context structure.

This structure is used by the driver for its internal logic. It must be provided by the application through the [FLEXIO_I2S_DRV_MasterInit\(\)](#) function, then it cannot be freed until the driver is de-initialized using [FLEXIO_I2S_DRV_MasterDeinit\(\)](#). The application should make no assumptions about the content of this structure.

Definition at line 156 of file [Flexio_I2s_Ip_Types.h](#).

6.2.2.3 struct Flexio_I2s_Ip_MasterConfigType

Master configuration structure.

This structure is used to provide configuration parameters for the flexio_i2s master at initialization time. Implements : [Flexio_I2s_Ip_MasterConfigType_Class](#)

Definition at line 190 of file [Flexio_I2s_Ip_Types.h](#).

Data Fields

Flexio_I2s_Ip_DriverType	driverType	Driver type: interrupts/polling/DMA
uint8	ResourceIndex	Index of first used internal resource instance (shifter and timer)
uint16	DividerValue	Calculated divider value
uint8	byteWidth	
uint8	txPin	Flexio pin to use for transmit

Data Fields

uint8	rxPin	Flexio pin to use for receive
uint8	sckPin	Flexio pin to use for serial clock
uint8	wsPin	Flexio pin to use for word select
Flexio_Mcl_Ip_PinPolarityType	pinPolarity	Flexio clock polarity used for data sampling
Flexio_I2s_CallbackType	txCallback	User Tx callback function. Note that this function will be called from the interrupt service routine, so its execution time should be as small as possible. It can be NULL if it is not needed
Flexio_I2s_CallbackType	rxCallback	User Rx callback function. Note that this function will be called from the interrupt service routine, so its execution time should be as small as possible. It can be NULL if it is not needed
void *	txCallbackParam	Parameter for the Tx callback function
void *	rxCallbackParam	Parameter for the Rx callback function
uint8	rxDMAChannel	Rx DMA channel number. Only used in DMA mode
uint8	txDMAChannel	Tx DMA channel number. Only used in DMA mode
Flexio_I2s_Ip_StateType *	MasterState	Pointer to master state

6.2.2.4 struct Flexio_I2s_Ip_SlaveConfigType

Slave configuration structure.

This structure is used to provide configuration parameters for the flexio_i2s slave at initialization time. Implements : Flexio_I2s_Ip_SlaveConfigType_Class

Definition at line 225 of file Flexio_I2s_Ip_Types.h.

Data Fields

Flexio_I2s_Ip_DriverType	driverType	Driver type: interrupts/polling/DMA
uint8	ResourceIndex	Index of first used internal resource instance (shifter and timer)
uint8	byteWidth	
uint8	txPin	Flexio pin to use for transmit
uint8	rxPin	Flexio pin to use for receive
uint8	sckPin	Flexio pin to use for serial clock
uint8	wsPin	Flexio pin to use for word select
Flexio_Mcl_Ip_PinPolarityType	pinPolarity	Flexio clock polarity used for data sampling
Flexio_I2s_CallbackType	txCallback	User Tx callback function. Note that this function will be called from the interrupt service routine, so its execution time should be as small as possible. It can be NULL if it is not needed
Flexio_I2s_CallbackType	rxCallback	User Rx callback function. Note that this function will be called from the interrupt service routine, so its execution time should be as small as possible. It can be NULL if it is not needed
void *	txCallbackParam	Parameter for the Tx callback function
void *	rxCallbackParam	Parameter for the Rx callback function

Data Fields

uint8	rxDMAChannel	Rx DMA channel number. Only used in DMA mode
uint8	txDMAChannel	Tx DMA channel number. Only used in DMA mode
Flexio_I2s_Ip_StateType *	SlaveState	Pointer to master state

6.2.3 Enum Reference

6.2.3.1 Flexio_I2s_Ip_StatusType

enum [Flexio_I2s_Ip_StatusType](#)

Status type Implements : Flexio_I2s_Ip_StatusType_Class.

Enumerator

FLEXIO_I2S_IP_STATUS_TX_UNDERRUN	TX underrun error
FLEXIO_I2S_IP_STATUS_RX_OVERRUN	RX overrun error
FLEXIO_I2S_IP_STATUS_ABORTED	A transfer was aborted

Definition at line 82 of file Flexio_I2s_Ip_Types.h.

6.2.3.2 Flexio_I2s_Ip_DriverType

enum [Flexio_I2s_Ip_DriverType](#)

Driver type: INTERRUPT/POLLING/DMA Implements : Flexio_I2s_Ip_DriverType_Class.

Enumerator

FLEXIO_I2S_IP_DRIVER_TYPE_INTERRUPT	Driver uses INTERRUPT for data transfers
FLEXIO_I2S_IP_DRIVER_TYPE_POLLING	Driver is based on POLLING
FLEXIO_I2S_IP_DRIVER_TYPE_DMA	Driver uses DMA for data transfers

Definition at line 97 of file Flexio_I2s_Ip_Types.h.

6.2.3.3 Flexio_I2s_Ip_EventType

enum [Flexio_I2s_Ip_EventType](#)

Enumerator

FLEXIO_I2S_IP_EVENT_RX_FULL	Rx buffer is full
FLEXIO_I2S_IP_EVENT_TX_EMPTY	Tx buffer is empty

Enumerator

FLEXIO_I2S_IP_EVENT_END_TRANSFER	The current transfer is ending. Only FLEXIO instance uses this event. The difference between this and event TX_EMPTY is: TX_EMPTY is generated when all data has been pushed to hardware fifo, users should not call DeInit here or some last data will be lost; END_TRANSFER is generated when all data has been pushed to line, the transmission will be stopped before users can start transmit again, user can call DeInit here. For receiving case, this event is the same as RX_FULL.
FLEXIO_I2S_IP_EVENT_ERROR	An error occurred during transfer

Definition at line 110 of file Flexio_I2s_Ip_Types.h.

6.2.4 Function Reference

6.2.4.1 Flexio_I2s_Ip_MasterInit()

```
void Flexio_I2s_Ip_MasterInit (
    uint8 Instance,
    uint8 Channel,
    const Flexio_I2s_Ip_MasterConfigType * ConfigPtr )
```

Initialize the FLEXIO_I2S master mode driver.

This function initializes the FLEXIO_I2S driver in master mode.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number
in	<i>ConfigPtr</i>	Pointer to the FLEXIO_I2S master user configuration structure. The function reads configuration data from this structure and initializes the driver accordingly. The application may free this structure after the function returns.

Returns

void.

6.2.4.2 Flexio_I2s_Ip_Deinit()

```
void Flexio_I2s_Ip_Deinit (
    uint8 Instance,
    uint8 Channel )
```

De-initialize the FLEXIO_I2S driver.

This function de-initializes the FLEXIO_I2S driver. The driver can't be used again until reinitialized.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number

Returns

void.

6.2.4.3 Flexio_I2s_Ip_SendData()

```
Flexio_I2s_Ip_StatusType Flexio_I2s_Ip_SendData (
    uint8 Instance,
    uint8 Channel,
    const uint8 * TxBuff,
    uint32 TxSize )
```

Perform a non-blocking send transaction on the I2S bus.

This function starts the transmission of a block of data and returns immediately. The rest of the transmission is handled by the interrupt service routine (if the driver is initialized in interrupt mode) or by the Flexio_I2s_Ip_GetStatus function (if the driver is initialized in polling mode). Use Flexio_I2s_Ip_GetStatus() to check the progress of the transmission.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number
in	<i>TxBuff</i>	Pointer to the data to be transferred
in	<i>TxSize</i>	Length in words of the data to be transferred. Word length is configured via byteWidth parameter in the init config structure.

Returns

Error or success status returned by API

6.2.4.4 Flexio_I2s_Ip_SendDataBlocking()

```
Flexio_I2s_Ip_StatusType Flexio_I2s_Ip_SendDataBlocking (
    uint8 Instance,
    uint8 Channel,
    const uint8 * TxBuff,
    uint32 TxSize,
    uint32 Timeout )
```

Perform a blocking send transaction on the I2S bus.

This function sends a block of data, and only returns when the transmission is complete.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number

Parameters

in	<i>TxBuff</i>	Pointer to the data to be transferred
in	<i>TxSize</i>	Length in words of the data to be transferred. Word length is configured via <i>byteWidth</i> parameter in the init config structure.
in	<i>Timeout</i>	Timeout for the transfer in milliseconds

Returns

Error or success status returned by API

6.2.4.5 Flexio_I2s_Ip_ReceiveData()

```
Flexio_I2s_Ip_StatusType Flexio_I2s_Ip_ReceiveData (
    uint8 Instance,
    uint8 Channel,
    uint8 * RxBuff,
    uint32 RxSize )
```

Perform a non-blocking receive transaction on the I2S bus.

This function starts the reception of a block of data and returns immediately. The rest of the reception is handled by the interrupt service routine (if the driver is initialized in interrupt mode) or by the `Flexio_I2s_Ip_GetStatus` function (if the driver is initialized in polling mode). Use `Flexio_I2s_Ip_GetStatus()` to check the progress of the reception.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number
out	<i>RxBuff</i>	Pointer to the buffer where to store received data
in	<i>RxSize</i>	Length in words of the data to be transferred. Word length is configured via <i>byteWidth</i> parameter in the init config structure.

Returns

Error or success status returned by API

6.2.4.6 Flexio_I2s_Ip_ReceiveDataBlocking()

```
Flexio_I2s_Ip_StatusType Flexio_I2s_Ip_ReceiveDataBlocking (
    uint8 Instance,
    uint8 Channel,
    uint8 * RxBuff,
    uint32 RxSize,
    uint32 Timeout )
```

Perform a blocking receive transaction on the I2S bus.

This function receives a block of data and only returns when the reception is complete.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number
out	<i>RxBuff</i>	Pointer to the buffer where to store received data
in	<i>RxSize</i>	Length in words of the data to be transferred. Word length is configured via <i>byteWidth</i> parameter in the init config structure.
in	<i>Timeout</i>	Timeout for the transfer in milliseconds

Returns

Error or success status returned by API

6.2.4.7 Flexio_I2s_Ip_TransferAbort()

```
Flexio_I2s_Ip_StatusType Flexio_I2s_Ip_TransferAbort (
    uint8 Instance,
    uint8 Channel )
```

Aborts a non-blocking I2S transaction.

This function aborts a non-blocking I2S transfer.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number

Returns

Error or success status returned by API

6.2.4.8 Flexio_I2s_Ip_GetStatus()

```
Flexio_I2s_Ip_StatusType Flexio_I2s_Ip_GetStatus (
    uint8 Instance,
    uint8 Channel,
    uint32 * BytesRemaining )
```

Get the status of the current non-blocking I2S transaction.

This function returns the current status of a non-blocking I2S transaction. A return code of FLEXIO_I2S_IP_ST↵ATUS_BUSY means the transfer is still in progress. Otherwise the function returns a status reflecting the outcome of the last transfer. When the driver is initialized in polling mode this function also advances the transfer by checking and handling the transmit and receive events, so it must be called frequently to avoid overflows or underflows.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number
out	<i>BytesRemaining</i>	Remaining number of bytes to be transferred

Returns

Error or success status returned by API

6.2.4.9 Flexio_I2s_Ip_SlaveInit()

```
void Flexio_I2s_Ip_SlaveInit (
    uint8 Instance,
    uint8 Channel,
    const Flexio_I2s_Ip_SlaveConfigType * ConfigPtr )
```

Initialize the FLEXIO_I2S slave mode driver.

This function initializes the FLEXIO_I2S driver in slave mode.

Parameters

in	<i>Instance</i>	FLEXIO peripheral instance number
in	<i>Channel</i>	FLEXIO_I2S logical channel number
in	<i>ConfigPtr</i>	Pointer to the FLEXIO_I2S slave user configuration structure. The function reads configuration data from this structure and initializes the driver accordingly. The application may free this structure after the function returns.

Returns

void.

6.2.4.10 Flexio_I2s_Ip_IrqHandler()

```
void Flexio_I2s_Ip_IrqHandler (
    uint8 FlexIOChannel,
    uint8 ShifterMaskFlag,
    uint8 ShifterErrMaskFlag,
    uint8 TimerMaskFlag )
```

Interrupt handler for FlexIO.

This function shall manage all the interrupts of a FlexIO module

Parameters

in	<i>FlexIOChannel</i>	FlexIO channel to be addressed.
in	<i>ShifterMaskFlag</i>	shifters status
in	<i>ShifterErrMaskFlag</i>	shifters error status
in	<i>TimerMaskFlag</i>	FlexIO timers status

Returns

void.

Note

Internal driver function.

6.3 SAI IPL

6.3.1 Detailed Description

Data Structures

- struct [Sai_Ip_XferState](#)
Transmit or receive state. [More...](#)
- struct [Sai_Ip_StateStructType](#)
Structure for internal use. This structure is used by the driver for its internal logic. It must be provided by the application through the initialize functions, then it cannot be freed until the driver is de-initialized using Deinit functions. The application should make no assumptions about the content of this structure. [More...](#)
- struct [Sai_Ip_ConfigType](#)
User config structure.
Note: entries in this structure are affected by [FEATURE_SAI_IP_HAS_CHMOD](#), which is device dependent and controlled from feature header file of the used device. [More...](#)

Macros

- `#define SAI_IP_FPACK_8_BIT`
- `#define SAI_IP_FPACK_16_BIT`
- `#define FEATURE_SAI_IP_HAS_CHMOD`
Channel mode tristate or output zero.
- `#define FEATURE_SAI_IP_MSEL_BUS_CLK`
Master clock from Bus clock.
- `#define FEATURE_SAI_IP_MSEL_SOSC_DIV1_CLK`
Master clock from SOSC_DIV1 clock.
- `#define FEATURE_SAI_IP_MSEL_MCLK_PIN`
Master clock from MCLK pin.
- `#define FEATURE_SAI_IP_MSEL_OTHER_MCLK_PIN`
Master clock from MCLK pin from other SAI instance.
- `#define FEATURE_SAI_IP_HAS_PARAM`
SAI has PARAM register.
- `#define SAI0_IP_CHANNEL_COUNT`
SAI channel count.
- `#define SAI_IP_NULL_PTRS`
SAI NULL PTRs.

Types Reference

- typedef void(* [Sai_Ip_WriteHandlerType](#)) (uint32 *const u32Data, const uint8 u8Instance, const uint8 u8←Pack, const uint8 u8ChnIdx)
Sai write data handler.
- typedef void(* [Sai_Ip_ReadHandlerType](#)) (uint32 *const u32Data, const uint8 u8Instance, const uint8 u8←ChnIdx)
Sai read data handler.
- typedef void(* [Sai_Ip_TransferCallbackType](#)) ([Sai_Ip_ReportType](#) event)

Sai callback function type for nonblock transfer, also called to report events (Sai_Ip_ReportType).

Enum Reference

- enum [Sai_Ip_ReportType](#)
Define the enum of the events which can trigger SAI callback.
- enum [Sai_Ip_StatusType](#)
SAI status return type.
- enum [Sai_Ip_TransferType](#)
Transfer type.
- enum [Sai_Ip_MuxType](#)
Data mux line or mux memory.
- enum [Sai_Ip_SyncType](#)
SAI run in sync or async mode.
Note: entries in this enum are affected by `::FEATURE_SAI_IP_SYNC_WITH_OTHER_INST`, which is device dependent and controlled from feature header file of the used device.
- enum [Sai_Ip_MasterClkSelType](#)
Select master clock.
- enum [Sai_Ip_MaskType](#)
Data line state for masked word, or if data line is disabled.

Function Reference

- void [Sai_Ip_SetMaster](#) (const uint8 u8Instance, const boolean bTx)
Set active for either transmitter or receiver.
- void [Sai_Ip_TxInit](#) (const uint8 u8Instance, const [Sai_Ip_ConfigType](#) *pConfig)
Initialize the transmitter of driver.
- void [Sai_Ip_RxInit](#) (const uint8 u8Instance, const [Sai_Ip_ConfigType](#) *pConfig)
Initialize the receiver of driver.
- void [Sai_Ip_TxDeInit](#) (const uint8 u8Instance)
De-initialize transmitter.
- void [Sai_Ip_RxDeInit](#) (const uint8 u8Instance)
De-initialize receiver.
- [Sai_Ip_StatusType](#) [Sai_Ip_SendBlocking](#) (const uint8 u8Instance, const uint8 *const aData[], const uint32 u32Count, const uint32 u32Timeout)
Send a block of data, return when transfer complete.
- void [Sai_Ip_Send](#) (const uint8 u8Instance, const uint8 *const aData[], const uint32 u32Count)
Send a block of data, return immediately.
- [Sai_Ip_StatusType](#) [Sai_Ip_GetSendingStatus](#) (const uint8 u8Instance, uint32 *pCountRemain)
Get status of a non-blocking transfer.
- [Sai_Ip_StatusType](#) [Sai_Ip_ReceiveBlocking](#) (const uint8 u8Instance, uint8 *const aData[], const uint32 u32Count, const uint32 u32Timeout)
Receive a block of data, return when transfer complete.
- void [Sai_Ip_Receive](#) (const uint8 u8Instance, uint8 *const aData[], const uint32 u32Count)

Receive a block of data, return immediately.

- [Sai_Ip_StatusType Sai_Ip_GetReceivingStatus](#) (const uint8 u8Instance, uint32 *pCountRemain)

Get status of a non-blocking transfer.

- void [Sai_Ip_AbortTransfer](#) (const uint8 u8Instance)

Abort all ongoing transferring data (both sending and receiving)

- void [Sai_Ip_IRQHandler](#) (const uint8 u8Instance)

Interrupt request handler for the SAI Ip.

- void [Sai_Ip_TxAutoDisableClock](#) (const uint8 u8Instance)

Disable transmitter clock after the transmission completed. This function has effect only for first call to Send function. Without calling this function, the clock will remain active after transmission has ended.

- void [Sai_Ip_RxAutoDisableClock](#) (const uint8 u8Instance)

Disable receiver clock after the transmission completed. This function has effect only for first call to Receive function. Without calling this function, the clock will remain active after transmission has ended.

6.3.2 Data Structure Documentation

6.3.2.1 struct Sai_Ip_XferState

Transmit or receive state.

Definition at line 227 of file Sai_Ip_Types.h.

6.3.2.2 struct Sai_Ip_StateStructType

Structure for internal use. This structure is used by the driver for its internal logic. It must be provided by the application through the initialize functions, then it cannot be freed until the driver is de-initialized using Deinit functions. The application should make no assumptions about the content of this structure.

Definition at line 242 of file Sai_Ip_Types.h.

6.3.2.3 struct Sai_Ip_ConfigType

User config structure.

Note: entries in this structure are affected by [FEATURE_SAI_IP_HAS_CHMOD](#), which is device dependent and controlled from feature header file of the used device.

Implements Sai_Ip_ConfigType_struct

Definition at line 283 of file Sai_Ip_Types.h.

Data Fields

Sai_Ip_SyncType	eSyncMode	Sync mode.
Sai_Ip_MasterClkSelType	eMasterClkSrc	Select master clock source.
boolean	bBitClkNegPolar	True if bit clock is negative polar (active low with drive outputs on falling edge and sample inputs on rising edge), false otherwise.
boolean	bBitClkInternal	True if bit clock is generated internally.
uint32	u32MasterClkFreq	Clock frequency from selected master clock source, only for internally generated master clock

Data Fields

uint32	u32BitClkFreq	Desired bit clock frequency in hertz, only for internally generated master clock and bit clock. Real divisor is derived by: $\text{floor}(\text{floor}(\text{MasterClockFreq} / \text{BitClkFreq}) / 2) * 2$, value range is from 2 to 512. If out of range value is assigned, maximum or minimum value is used. Real closest BitClockFreq is derived by: $\text{MasterClockFreq} / \text{real_divisor}$
uint8	u8ChannelEnable	Turn on each bit to enable each channel. 4 bit for 4 channels.
uint8	u8FrameSize	Frame size in number of words. When in mux line mode, the number of unmasked word in a frame must be divisible by number of channels
uint8	u8SyncWidth	Active sync width in number of bit clocks (must not longer than first word width).
Sai_Ip_MaskType	eMaskMode	Data line state for mask word or when data line is disabled (apply only for transmitter).
boolean	bDebugEnabled	True if transfer is enabled in debug mode after completing the current frame, false if transfer is disabled in debug mode.
boolean	bMsbFirst	True if data is MSB first, false if LSB first.
boolean	bSyncEarly	True if frame sync is one bit clock early.
boolean	bSyncNegPolar	True if frame sync is negative polar (active low), false otherwise.
boolean	bSyncInternal	True if frame sync is generated internally
uint8	u8Word0Width	First word width in number of bit clocks.
uint8	u8WordNWidth	Other words width in number of bit clocks.
uint8	u8FirstBitIndex	Index from LSB of first bit to be transmitted/received, valid range from 0-31.
boolean	bRunErrorReport	Underrun/overflow error report.
boolean	bSyncErrorReport	Enable sync error report.
boolean	bFrameStartReport	Enable frame start report.
boolean	bFPack	FIFO packing enable.
Sai_Ip_MuxType	eMuxMode	Enable line mux, memory mux or mux is disabled.
Sai_Ip_TransferType	eTransferType	Transfer using dma or interrupt.
uint8	aDmaChannel[4U]	DMA channels to be used.
uint8	u8ElementSize	Size in bytes of each read/write from/to buffer.
uint8	u8ChannelCount	Number of channels to enable, only used when both line mux mode and interrupt mode is selected.
uint8	u8BufferAccessSize	Number of words each time read from/write to buffer
uint8	u8WordFlagIndex	Configure which word sets the start of word flag, should less than frame size
boolean	bBitClkInput	Use slave mode timing in datasheet for master mode
boolean	bContOnErr	True continue transferring the same work that caused FIFO error, False start the next frame
boolean	bSyncOnDemand	True frame sync is generated only when fifo is not empty (transmit) or not full (receive), False otherwise

Data Fields

Sai_Ip_TransferCallbackType	pTxCallback	User Tx callback function, called when transfer complete or selected events occurred.
Sai_Ip_TransferCallbackType	pRxCallback	User Rx callback function, called when transfer complete or selected events occurred.
Sai_Ip_StateStructType *	pState	Sai IP internal state.

6.3.3 Macro Definition Documentation

6.3.3.1 SAI_IP_FPACK_8_BIT

```
#define SAI_IP_FPACK_8_BIT
```

8-bit FIFO packing is enabled

Definition at line 80 of file Sai_Ip_FeatureDefines.h.

6.3.3.2 SAI_IP_FPACK_16_BIT

```
#define SAI_IP_FPACK_16_BIT
```

16-bit FIFO packing is enabled

Definition at line 81 of file Sai_Ip_FeatureDefines.h.

6.3.3.3 FEATURE_SAI_IP_HAS_CHMOD

```
#define FEATURE_SAI_IP_HAS_CHMOD
```

Channel mode tristate or output zero.

Definition at line 83 of file Sai_Ip_FeatureDefines.h.

6.3.3.4 FEATURE_SAI_IP_MSEL_BUS_CLK

```
#define FEATURE_SAI_IP_MSEL_BUS_CLK
```

Master clock from Bus clock.

Definition at line 85 of file Sai_Ip_FeatureDefines.h.

6.3.3.5 FEATURE_SAI_IP_MSEL_SOSC_DIV1_CLK

```
#define FEATURE_SAI_IP_MSEL_SOSC_DIV1_CLK
```

Master clock from SOSC_DIV1 clock.

Definition at line 87 of file Sai_Ip_FeatureDefines.h.

6.3.3.6 FEATURE_SAI_IP_MSEL_MCLK_PIN

```
#define FEATURE_SAI_IP_MSEL_MCLK_PIN
```

Master clock from MCLK pin.

Definition at line 89 of file Sai_Ip_FeatureDefines.h.

6.3.3.7 FEATURE_SAI_IP_MSEL_OTHER_MCLK_PIN

```
#define FEATURE_SAI_IP_MSEL_OTHER_MCLK_PIN
```

Master clock from MCLK pin from other SAI instance.

Definition at line 91 of file Sai_Ip_FeatureDefines.h.

6.3.3.8 FEATURE_SAI_IP_HAS_PARAM

```
#define FEATURE_SAI_IP_HAS_PARAM
```

SAI has PARAM register.

Definition at line 93 of file Sai_Ip_FeatureDefines.h.

6.3.3.9 SAI0_IP_CHANNEL_COUNT

```
#define SAI0_IP_CHANNEL_COUNT
```

SAI channel count.

Definition at line 95 of file Sai_Ip_FeatureDefines.h.

6.3.3.10 SAI_IP_NULL_PTRS

```
#define SAI_IP_NULL_PTRS
```

SAI NULL PTRs.

Definition at line 101 of file Sai_Ip_FeatureDefines.h.

6.3.4 Types Reference

6.3.4.1 Sai_Ip_WriteHandlerType

```
typedef void(* Sai_Ip_WriteHandlerType) (uint32 *const u32Data, const uint8 u8Instance, const uint8 u8←  
Pack, const uint8 u8ChnIdx)
```

Sai write data handler.

Definition at line 211 of file Sai_Ip_Types.h.

6.3.4.2 Sai_Ip_ReadHandlerType

```
typedef void(* Sai_Ip_ReadHandlerType) (uint32 *const u32Data, const uint8 u8Instance, const uint8 u8ChnIdx)
```

Sai read data handler.

Definition at line 215 of file Sai_Ip_Types.h.

6.3.4.3 Sai_Ip_TransferCallbackType

```
typedef void(* Sai_Ip_TransferCallbackType) (Sai_Ip_ReportType event)
```

Sai callback function type for nonblock transfer, also called to report events (Sai_Ip_ReportType).

Definition at line 219 of file Sai_Ip_Types.h.

6.3.5 Enum Reference

6.3.5.1 Sai_Ip_ReportType

```
enum Sai_Ip_ReportType
```

Define the enum of the events which can trigger SAI callback.

This enum should include the events for all platforms

Enumerator

SAI_IP_RX_COMPLETE	Rx transfer complete, user can start another transfer
SAI_IP_TX_COMPLETE	Tx transfer complete, user can start another transfer
SAI_IP_ERROR	DMA error while transfer
SAI_IP_FRAME_START	Indicate a frame start
SAI_IP_RUN_ERROR	Overrun/underrun error. This event is useful for tx when user want to know whether all data in tx fifo has been pushed out to line and tx deinit can be called, since tx complete event only indicates all data from buffer has been pushed to hardware fifo
SAI_IP_SYNC_ERROR	Frame sync error

Definition at line 111 of file Sai_Ip_Types.h.

6.3.5.2 Sai_Ip_StatusType

```
enum Sai_Ip_StatusType
```

SAI status return type.

This structure is used as return type

Enumerator

SAI_IP_STATUS_UNINITIALIZED	Function unsupported
-----------------------------	----------------------

Enumerator

SAI_IP_STATUS_COMPLETED	Function completed successfully
SAI_IP_STATUS_ERROR	Function didn't complete successfully
SAI_IP_STATUS_TIMEOUT	Function timed out
SAI_IP_STATUS_BUSY	Function busy
SAI_IP_STATUS_ABORTED	Function aborted

Definition at line 128 of file Sai_Ip_Types.h.

6.3.5.3 Sai_Ip_TransferType

enum [Sai_Ip_TransferType](#)

Transfer type.

Enumerator

SAI_IP_INTERRUPT	Transfer type is interrupt
SAI_IP_DMA	Transfer type is DMA

Definition at line 140 of file Sai_Ip_Types.h.

6.3.5.4 Sai_Ip_MuxType

enum [Sai_Ip_MuxType](#)

Data mux line or mux memory.

Enumerator

SAI_IP_MUX_DISABLED	Each data line is a channel, uses a separate memory block
SAI_IP_MUX_LINE	Only one data line (data line 0 for DMA mode) is used. Words on data line is alternated between channels, each channel data is a separate memory block
SAI_IP_MUX_MEM	Words in memory block is alternated between channels, each channel data is on a separate data line.

Definition at line 150 of file Sai_Ip_Types.h.

6.3.5.5 Sai_Ip_SyncType

enum [Sai_Ip_SyncType](#)

SAI run in sync or async mode.

Note: entries in this enum are affected by ::FEATURE_SAI_IP_SYNC_WITH_OTHER_INST, which is device dependent and controlled from feature header file of the used device.

Enumerator

SAI_IP_ASYNC	Independent clock
SAI_IP_SYNC_WITH_OTHER	Bit clock and frame sync signal is taken from transmitter/receiver

Definition at line 161 of file Sai_Ip_Types.h.

6.3.5.6 Sai_Ip_MasterClkSelType

enum `Sai_Ip_MasterClkSelType`

Select master clock.

Enumerator

SAI_IP_BUS_CLK	Master clock is module Bus clock
SAI_IP_EXTERNAL_CLK	Master clock is from external
SAI_IP_SOSC_DIV1_CLK	Master clock is from external oscillator/crystal
SAI_IP_EXTERNAL_OTHER_CLK	Master clock is from external other sai instance

Definition at line 177 of file Sai_Ip_Types.h.

6.3.5.7 Sai_Ip_MaskType

enum `Sai_Ip_MaskType`

Data line state for masked word, or if data line is disabled.

Enumerator

SAI_IP_MASK_TRISTATE	Line is in high z state
SAI_IP_MASK_ZERO	Line is output zero

Definition at line 202 of file Sai_Ip_Types.h.

6.3.6 Function Reference

6.3.6.1 Sai_Ip_SetMaster()

```
void Sai_Ip_SetMaster (
    const uint8 u8Instance,
    const boolean bTx )
```

Set active for either transmitter or receiver.

When both of transmitter and receiver are configured as master and async mode, at the specific time only one master can be active. This function must be called to alternate between sending and receiving operation:

- Before calling the first tx/rx transfer

- After a tx/rx transfer end and before start another rx/tx transfer.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
in	<i>bTx</i>	True if next operation is tx, false otherwise

6.3.6.2 Sai_Ip_TxInit()

```
void Sai_Ip_TxInit (
    const uint8 u8Instance,
    const Sai_Ip_ConfigType * pConfig )
```

Initialize the transmitter of driver.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
in	<i>pConfig</i>	Pointer to the user configuration structure. The function reads configuration data from this structure and initializes the driver accordingly. The application may free this structure after the function returns.

6.3.6.3 Sai_Ip_RxInit()

```
void Sai_Ip_RxInit (
    const uint8 u8Instance,
    const Sai_Ip_ConfigType * pConfig )
```

Initialize the receiver of driver.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
in	<i>pConfig</i>	Pointer to the user configuration structure. The function reads configuration data from this structure and initializes the driver accordingly. The application may free this structure after the function returns.

6.3.6.4 Sai_Ip_TxDeInit()

```
void Sai_Ip_TxDeInit (
    const uint8 u8Instance )
```

De-initialize transmitter.

This function de-initializes driver. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
----	-------------------	----------------------------

6.3.6.5 Sai_Ip_RxDeInit()

```
void Sai_Ip_RxDeInit (
    const uint8 u8Instance )
```

De-initialize receiver.

This function de-initializes driver. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
----	-------------------	----------------------------

6.3.6.6 Sai_Ip_SendBlocking()

```
Sai_Ip_StatusType Sai_Ip_SendBlocking (
    const uint8 u8Instance,
    const uint8 *const aData[],
    const uint32 u32Count,
    const uint32 u32Timeout )
```

Send a block of data, return when transfer complete.

Should be called immediately after a transfer complete to avoid data underrun error.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
in	<i>aData</i>	Array of pointer to each data block to transfer, each data block corresponds to an enabled channels If mux memory is selected, only first data block is used
in	<i>u32Count</i>	Number of words to transfer for each channel. The size of each buffer word element is "Sai Buffer Element Size" selected in configurator. In DMA mode, count number upper limit is limited by DMA BITER/CITER register length.
in	<i>u32Timeout</i>	Timeout to return when transfer take too long.

Returns

Success, error or timeout status.

6.3.6.7 Sai_Ip_Send()

```
void Sai_Ip_Send (
    const uint8 u8Instance,
```

```
const uint8 *const aData[],
const uint32 u32Count )
```

Send a block of data, return immediately.

When transfer completed, the callback function will be executed. User should use this callback function to immediately start an other transfer to avoid data underrun error.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
in	<i>aData</i>	Array of pointer to each data block to transfer, each data block corresponds to an enabled channels If mux memory is selected, only first data block is used
in	<i>u32Count</i>	Number of words to transfer for each channel. The size of each buffer word element is "Sai Buffer Element Size" selected in configurator. In DMA mode, count number upper limit is limited by DMA BITER/CITER register length.

6.3.6.8 Sai_Ip_GetSendingStatus()

```
Sai_Ip_StatusType Sai_Ip_GetSendingStatus (
    const uint8 u8Instance,
    uint32 * pCountRemain )
```

Get status of a non-blocking transfer.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
out	<i>pCountRemain</i>	Number of words remain for each channel. This parameter can be NULL

Returns

Status of the transfer, can be success, aborted, timeout or busy. Note that for tx, success status imply that all data has been pushed to hardware fifo and another transfer can be started.

6.3.6.9 Sai_Ip_ReceiveBlocking()

```
Sai_Ip_StatusType Sai_Ip_ReceiveBlocking (
    const uint8 u8Instance,
    uint8 *const aData[],
    const uint32 u32Count,
    const uint32 u32Timeout )
```

Receive a block of data, return when transfer complete.

Should be called immediately after a transfer complete to avoid data overrun error.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
out	<i>aData</i>	Array of pointer to each data block to transfer, each data block corresponds to an enabled channels If mux memory is selected, only first data block is used

Parameters

in	<i>u32Count</i>	Number of words to transfer for each channel. The size of each buffer word element is "Sai Buffer Element Size" selected in configurator. In DMA mode, count number upper limit is limited by DMA BITER/CITER register length.
in	<i>u32Timeout</i>	Timeout to return when transfer take too long.

Returns

Success, error or timeout status.

6.3.6.10 Sai_Ip_Receive()

```
void Sai_Ip_Receive (
    const uint8 u8Instance,
    uint8 *const aData[],
    const uint32 u32Count )
```

Receive a block of data, return immediately.

When transfer completed, the callback function will be executed. User should use this callback function to immediately start another transfer to avoid data overrun error.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
out	<i>aData</i>	Array of pointer to each data block to transfer, each data block corresponds to an enabled channels If mux memory is selected, only first data block is used
in	<i>u32Count</i>	Number of words to transfer for each channel. The size of each buffer word element is "Sai Buffer Element Size" selected in configurator. In DMA mode, count number upper limit is limited by DMA BITER/CITER register length.

6.3.6.11 Sai_Ip_GetReceivingStatus()

```
Sai_Ip_StatusType Sai_Ip_GetReceivingStatus (
    const uint8 u8Instance,
    uint32 * pCountRemain )
```

Get status of a non-blocking transfer.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
out	<i>pCountRemain</i>	Number of words remain for each channel. This parameter can be NULL

Returns

Status of the transfer, can be success, aborted, timeout or busy.

6.3.6.12 Sai_Ip_AbortTransfer()

```
void Sai_Ip_AbortTransfer (
    const uint8 u8Instance )
```

Abort all ongoing transferring data (both sending and receiving)

Parameters

in	<i>u8Instance</i>	Peripheral instance number
----	-------------------	----------------------------

6.3.6.13 Sai_Ip_IRQHandler()

```
void Sai_Ip_IRQHandler (
    const uint8 u8Instance )
```

Interrupt request handler for the SAI Ip.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
----	-------------------	----------------------------

6.3.6.14 Sai_Ip_TxAutoDisableClock()

```
void Sai_Ip_TxAutoDisableClock (
    const uint8 u8Instance )
```

Disable transmitter clock after the transmission completed. This function has effect only for first call to Send function. Without calling this function, the clock will remain active after transmission has ended.

Parameters

in	<i>u8Instance</i>	Peripheral instance number
----	-------------------	----------------------------

6.3.6.15 Sai_Ip_RxAutoDisableClock()

```
void Sai_Ip_RxAutoDisableClock (
    const uint8 u8Instance )
```

Disable receiver clock after the transmission completed. This function has effect only for first call to Receive function. Without calling this function, the clock will remain active after transmission has ended.

Module Documentation

Parameters

in	<i>u8Instance</i>	Peripheral instance number
----	-------------------	----------------------------

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.